Kubernetes, GitOps, and All That

New Tricks for Old Dogs

Will Ware

September 9, 2026

Contents

1 From Pet Servers to Cattle **1**
SSH in, hand-edit, restart, hope . . . 1
Snowflakes, and the gap between "works" and "works reliably" . . . 1
Describe, don't do . . . 1
Physical servers to orchestrated containers, briefly . . . 2

2 The Security Case for Systematized Infrastructure **3**
Ad-hoc ops was always risky . . . 3
Why it's worse now . . . 3
No diff, no review, no rollback . . . 3
If it's not in git, it shouldn't be running . . . 4

3 Docker: Packaging Reality **5**
What a container actually is, briefly . . . 5
Images vs. containers, and the Dockerfile as a recipe . . . 5
A short history, and why "works in the container" is a stronger claim . . . 6
Further reading . . . 6

4 Docker Compose: Orchestration's Training Wheels **7**
One file, two services, one command . . . 7
What Compose gets right . . . 8
Where the ceiling actually is . . . 8
The right tool until it isn't . . . 8

5 What Kubernetes Actually Is **11**
The control loop, not the orchestrator . . . 11
The control plane is not magic – it's pods . . . 11
Mapping what you already know . . . 12

6 Your First Deployment **13**
`./start.sh`: containers + orchestration, not magic . . . 13
Deployments, Services, and the separation of "what runs" from "how it's reached" . . . 13
StatefulSets, and why databases aren't just "a Deployment with a volume" . . . 14
Hands-on: deploying the toy API + Postgres from this repo . . . 15
The YAML files as promises kept . . . 16

7 Self-Healing and Scaling **17**
Deleting a pod on purpose and watching Kubernetes notice . . . 17
Declarative intent: a standing order, not a one-time command . . . 17
Scaling out: easy for the API, meaningless for the database . . . 17
Load balancing across replicas, observed via logs . . . 18

8 Configuration, Secrets, and Storage **21**
Editing a ConfigMap live, and finding the edge of "hot reload" . . . 21
PersistentVolumeClaims: storage that outlives the pod . . . 22
ConfigMap and Secret, side by side . . . 22

9 When Not to Use Kubernetes **25**
The same app, two ways . . . 25
What all that extra machinery is buying, here, right now . . . 25
What running Kubernetes well actually requires . . . 26
Signs you don't need it yet, and signs you're about to . . . 26

10 Infrastructure as Code, Take One: Pulumi **27**
Same resources, different syntax . . . 27
Type checking and reusable modules . . . 28
Where this file actually stands right now . . . 28
When Pulumi earns its complexity over plain manifests . . . 28

11 Observability Basics: Logging **29**
One line becomes three, and a pod name you didn't ask for . . . 29
The line the endpoint wrote vs. the line the formatter added . . . 30
Where the trail actually ends . . . 30

12 Authentication and Authorization Patterns **33**
Two questions that sound like one . . . 33
The application side: an API key, actually wired in . . . 33
The cluster side: RBAC, checked directly . . . 34
On AWS: the same ServiceAccount does more work . . . 35
Where each one actually belongs . . . 35

13 Git as the Control Plane **37**
The inversion . . . 37
ArgoCD's loop is the same loop . . . 37
Hands-on: this repo, deployed by ArgoCD, for real . . . 37
What "the cluster is passive" actually buys . . . 38

14 Drift Detection and Self-Healing at the Fleet Level **41**
Two settings, two very different outcomes . . . 41
The diff is there, even when nothing acts on it . . . 41
A permanent diff, not a one-time audit . . . 42
Even the control file can drift . . . 42

15 Multi-Environment Deployment Without the Copy-Paste **45**
Three environments, one template . . . 45
What actually differs between them . . . 45
Why hand-maintained per-environment YAML rots . . . 46

16 Autoscaling on Real Signal: KEDA **49**
What CPU can't see . . . 49
Watching it scale from zero, for real . . . 49
Scaling back to zero, and paying for the gap honestly . . . 50

17 The Cost/Complexity Decision **51**
The same problem, three price points . . . 51
Option A: single-box autoscaling . . . 51
Option B: AWS Auto Scaling Groups . . . 51
Option C: real EKS . . . 52
A decision framework, not a default answer . . . 52

18 Side Quests **53**
EKS emulation on a home LAN . . . 53
Queue-based scaling as a portable pattern . . . 53

19 GitOps Without a Cluster, Watched Live **55**
What Kubernetes was quietly supplying for free . . . 55
Watching the loop notice a change, live . . . 55
A worthwhile honest gap, found by accident . . . 56
The shared shape underneath all of it . . . 56

20 Design Decisions, and Why They Were Made That Way **57**
Why an ABC instead of a Protocol, checked live . . . 57
Why there's no `plan()` or dry-run method . . . 58
Why there's no built-in approval gate – and where it actually goes . . . 58
Why pull-vs-push and convergence stay on separate sides . . . 58
What actually ran, and what's sketched . . . 58

21 Credentials and Blast Radius **61**
The reconciler's environment is the actual security boundary . . . 61
Short-lived over static, for the same reason Chapter 12 gave . . . 61
The exception that proves the rule . . . 61
The same split as Chapter 12, one layer up . . . 62

22 Progressive Delivery Without New Abstractions **63**
Two targets, one repo, no new machinery . . . 63
Watching a real promotion happen . . . 63
Two parallels, not one . . . 64
The honest gap, demonstrated . . . 64

23 One Pattern, Three Substrates **67**
The same three verbs, every time . . . 67
What each substrate actually buys, side by side . . . 67
A checklist for a new project . . . 68

24 What Still Isn't Covered **69**
The honest list . . . 69
Where to actually go next . . . 69

A Command Reference **71**
Docker / Docker Compose . . . 71
kubectl: pods, deployments, and workloads . . . 71
kubectl: config, secrets, RBAC . . . 72
kubectl: ArgoCD, KEDA, and cluster inspection . . . 72
Git . . . 72
Pulumi . . . 72
gitops_reconciler . . . 73

B Glossary **75**

C Repository Map **77**

Chapter 1

From Pet Servers to Cattle

SSH in, hand-edit, restart, hope

The old workflow doesn't need much reconstruction because most people who'd pick up this book have lived some version of it: `ssh` into the box, `vim` the config file, restart the service, watch the logs scroll by for a minute to confirm nothing's on fire, log out. It works. It's also the entire deployment process, in the sense that nothing about it exists anywhere except in that terminal session and, if you're lucky, in whatever you remember about it later.

That's the part worth sitting with, not the SSH command itself. The change you just made – what config value moved, why, what it was before – lives in exactly one place: your memory of doing it, maybe backed by a comment you left in the file, if you left one. Six months later, "why is this set to 30 instead of the default of 10" has one honest answer: ask whoever did it, if they still work there, if they remember.

Snowflakes, and the gap between "works" and "works reliably"

Do this enough times, on enough servers, and every server drifts into being slightly different from every other one – not because anyone planned it that way, but because each one accumulated its own particular sequence of hand-applied fixes, each one applied under time pressure, to whatever that specific box happened to need at whatever moment someone was looking at it. Server A got a kernel parameter tuned during an incident eighteen months ago. Server B didn't, because it wasn't part of that incident. Nobody wrote either change down anywhere both servers would show up.

"It works on my machine" is the famous version of this joke, but the sharper version, and the one that actually costs money, is "it works on this production server" – meaning specifically this one, the one that's been hand-tuned by three different people responding to three different emergencies, and not the one you just brought up from the same base image that's supposedly identical to it. Two servers built from the same starting point stop being identical the first time someone touches one of them by hand and not the other. There's no mechanism forcing them back into agreement, so unless someone is deliberately auditing for drift – and auditing for drift by hand doesn't scale past a small number of servers – they just keep diverging.

Describe, don't do

Puppet, Chef, and Ansible were the first widely-adopted answer to this, and the shift they represent is worth naming precisely, because it's the same shift Kubernetes makes later, just at a smaller scope. The old workflow is a sequence of commands: SSH in, run this, run that, check if it worked. A Puppet manifest or an Ansible playbook is a description of what the machine should look like – this package installed, this file containing this content, this service running – and the tool's job is to look at the machine, compare it to the description, and make only the changes needed to close the gap. Run the same playbook twice and the second run should do nothing, because the machine already matches the description. That property – safe to reapply, because it only acts on the difference – is the whole reason these tools were an improvement, and it's the same property Chapter 5 is going to name explicitly as a control loop.

It wasn't a complete fix. These tools still ran on a schedule, or on demand, not continuously, so drift could reopen between runs, and someone still had to remember to run them. But the core move – write down what the machine

should look like, then let a tool make it true – is the same move every chapter after this one keeps making at a larger scope.

Physical servers to orchestrated containers, briefly

The rest of this progression is really the same idea, applied one layer up each time, as the unit being managed keeps shrinking and multiplying: physical servers you could touch, one OS per machine, one workload's mistake capable of taking down everything else sharing that hardware. Virtual machines split one physical box into several isolated ones, solving the sharing problem but still booting a full OS per workload, still slow to provision, still something you patched and drifted the same way individual servers had. Containers dropped the "boot a full OS" requirement – share the host kernel, isolate everything else – and suddenly a workload went from something that took minutes to provision to something that took seconds, and from one process per physical or virtual machine to dozens of processes safely sharing one machine.

That last shift is what makes orchestration necessary rather than optional. Provisioning a VM was slow and rare enough that a human deciding where it should run was fine. Once workloads are containers that start in under a second and a single host might run dozens of them, "a person decides where each one goes" stops being a workflow and starts being the bottleneck. Something has to decide placement, notice failures, and keep desired counts correct, continuously, faster than any person could do it by hand. That something is the subject of the rest of this book.

Chapter 2

The Security Case for Systematized Infrastructure

Ad-hoc ops was always risky

Chapter 1 made the productivity case against hand-edited servers: drift, lost context, nobody quite sure what's actually running. All of that is also, separately, a security problem, and it was one even before the threat landscape got worse. A server that's been hand-patched by three different people over two years has no record of what was changed, which means it also has no record of *whether every change was supposed to happen*. An unauthorized change and an authorized-but-undocumented change look identical from the outside – neither one shows up anywhere except in what the server is currently doing.

Why it's worse now

Two things changed the stakes on top of that baseline risk. First, attack tooling industrialized. Scanning the entire public IPv4 address space for a specific vulnerable service configuration used to be a research project; it's now a background process, running constantly, from multiple directions at once. A misconfiguration that used to be safe because nobody was likely to stumble onto it in the time it took you to notice and fix it is no longer safe on that assumption – something is checking, right now, and will check again in an hour.

Second, the supply chain became a target in its own right. A compromised build dependency, a poisoned base image, a maintainer's stolen credentials on a widely-used package – these don't require finding a hole in your infrastructure at all. They ride in through the normal process of building and deploying software, the same process every other chapter in this book is about making safer, not riskier. Ransomware-as-a-service turned the profit motive behind all of this into something available to anyone willing to pay for access, not just a small number of technically sophisticated actors. The threat model isn't "a skilled attacker might target us specifically" anymore. It's "automated tooling will find whatever's exposed, and someone downstream will monetize it."

No diff, no review, no rollback

Put those two together and the hand-edited server from Chapter 1 stops being merely inefficient and starts being a liability, for a specific, mechanical reason: a manual change has no diff. Nobody reviewed it before it went live, because there was no artifact to review – the "change" was a person typing commands into a terminal, not a pull request. If it turns out to have been a mistake, or worse, if it turns out to have been made by someone who shouldn't have had that access in the first place, there's no clean way to know what it changed or to undo it, because undoing it means remembering, by hand, what it was before.

Version-controlled infrastructure closes that gap by construction, not by policy. When the desired state of a server or a cluster lives in a git repository, every change is a commit: who made it, when, exactly what changed, and – if the review discipline from ordinary software engineering gets applied here too – someone else looked at it before it took effect. A bad change is `git revert`, not an afternoon of archaeology. This isn't a new security control bolted onto infrastructure that used to lack one. It's the existing discipline of code review and version control, already trusted for the application, extended to cover the infrastructure that application runs on.

If it's not in git, it shouldn't be running

That's the principle worth carrying into every chapter after this one, because it's going to come back explicitly more than once: version control isn't a convenience for infrastructure, it's the mechanism that makes infrastructure auditable at all. A system where every running change traces back to a reviewed commit is a system where "what's running and why" always has an answer. A system where changes can still be made by hand, outside that record, has a permanent, unfixable gap between what the repository says and what's actually true – and that gap is exactly where both Chapter 1's drift problem and this chapter's security problem live. Chapter 14 is going to spend real time on what happens once that gap gets automated away entirely, git no longer just describing infrastructure but actively defending it. Everything between here and there is really this same idea, worked out at increasing scale.

Chapter 3

Docker: Packaging Reality

What a container actually is, briefly

A container is not a lightweight virtual machine, even though it gets described that way often enough that the description sticks. A VM virtualizes hardware and boots a full second kernel on top of it. A container is just an ordinary process on the host, running under the same kernel as everything else, made to believe it's alone through three mechanisms working together: **namespaces** hide everything the process shouldn't see – its own process ID space, its own network interfaces, its own filesystem mounts, so `ps` inside the container shows a handful of processes, not the host's real few hundred. **cgroups** cap what the process is allowed to consume – CPU, memory, I/O – so one runaway container can't starve everything else on the box. And a **union filesystem** layers a stack of read-only image layers under one writable layer on top, so the container appears to have its own private filesystem without needing to actually copy the whole thing.

None of that is magic, and none of it requires believing anything about containers being a fundamentally new kind of computing. It's namespacing, resource limiting, and clever filesystem layering, and Docker's actual contribution in 2013 wasn't inventing any of these three mechanisms – Linux had namespaces and cgroups already, and LXC had been wiring them together for years before Docker existed. Docker's contribution was making the packaging and distribution of the result trivial: a `Dockerfile`, a build command, and an image anyone else can pull and run without caring how any of those three mechanisms actually work.

Images vs. containers, and the Dockerfile as a recipe

The distinction that trips people up first: an image is not a container, it's what a container is made from. This repo's `Dockerfile` is short enough to read as a whole:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py ./

EXPOSE 8000
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

Six meaningful lines, and each one – `FROM`, `COPY`, `RUN`, `COPY` again – adds one more layer to the union filesystem the previous section described. `docker build` doesn't run this file the way a shell script runs; it produces an image, a stack of those layers, and it's worth seeing that stack for real rather than taking "layered" as an abstract claim:

```
docker history k8s-toy-api:local
```

```
IMAGE          CREATED        CREATED BY                                      SIZE
57163a50e75e   ...            CMD ["uvicorn" "app:app" ...                    0B
<missing>      ...            EXPOSE [8000/tcp]                               0B
<missing>      ...            COPY app.py ./ # buildkit                       11.3kB
<missing>      ...            RUN /bin/sh -c pip install --no-cache-dir -r…   62.1MB
```

```
<missing>      ...           COPY requirements.txt ./ # buildkit            115B
<missing>      ...           WORKDIR /app                                   0B
                             [... python:3.12-slim base layers below ...]
```

`COPY app.py ./` is 11.3 kilobytes – that's the entire application, one file. Everything above a few hundred kilobytes in that list belongs to either the `pip install` layer or the Debian base image underneath it, none of which this repo wrote. That's the actual payoff of layering, made concrete instead of asserted: change `app.py` and rebuild, and Docker only has to redo the `COPY app.py` layer and whatever comes after it in the file – the `pip install` layer, unchanged since `requirements.txt` didn't change, gets reused straight from cache. A Dockerfile is a recipe in the specific sense that each step describes an incremental change to apply on top of the last one, not a script that runs top to bottom and discards its intermediate state.

A short history, and why "works in the container" is a stronger claim

`chroot` gave a process its own root filesystem view in 1979 – the oldest of these three mechanisms by a wide margin, and proof this idea isn't new. LXC, starting around 2008, was the first attempt to wire namespaces and cgroups together into something usable as "a container," and it worked, but using it meant understanding all three mechanisms individually and assembling them by hand. Docker's 2013 release didn't replace any of that machinery – early Docker used LXC directly under the hood – it replaced the assembly step with a `Dockerfile` and a single `docker build`, and that packaging leap is what actually took off. The OCI (Open Container Initiative) standardization that followed formalized the image format itself, which is why an image built by Docker runs fine under containerd or Podman today – the format outlived the tool that popularized it.

"Works on my machine" was never a claim about the code; it was a claim about everything *surrounding* the code on that one machine – library versions, OS packages, environment variables nobody wrote down. "Works in the container" is a stronger claim because the image is that entire surrounding environment, made explicit in a file, checked into git next to the code it runs. `python:3.12-slim` at the base of this repo's image is a specific, named, versioned thing, not "whatever Python happens to be installed on this box today." The whole rest of this book is going to keep leaning on that same move – take something that used to live only in one person's memory of what they did to a machine, and make it a committed file instead – so it's worth having Chapter 3 be the place that move first gets named plainly, at the smallest possible scale, one image.

Further reading

Docker's own "Get Started" guide is still the fastest way to build the muscle memory for `build`/`run`/`exec` before any of the orchestration chapters pile more on top of it. *The Docker Book* (James Turnbull) goes deeper into the daemon and networking model than this chapter needs to. The OCI image spec itself, for anyone who wants to see exactly what "layer" and "manifest" mean at the level of actual JSON on disk, is short enough to read in one sitting and worth it once namespaces and cgroups stop being new.

Chapter 4

Docker Compose: Orchestration's Training Wheels

One file, two services, one command

`docker-compose.yml` in this repo describes the same two-service app Chapter 3 built one image for – `postgres` and `api` – as a single YAML file instead of two separate `docker run` invocations someone would otherwise have to remember and re-type correctly every time:

```yaml
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: toyapi
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 3s
      retries: 5

  api:
    build:
      context: .
      dockerfile: Dockerfile
    environment:
      DATABASE_URL: "postgresql://postgres:postgres@postgres:5432/toyapi"
    depends_on:
      postgres:
        condition: service_healthy
```

`docker compose up -d --build` builds the image from Chapter 3's Dockerfile, starts both containers, and waits:

```
docker compose up -d --build
```

```
 Container k8s-hack-postgres-1  Starting
 Container k8s-hack-postgres-1  Started
 Container k8s-hack-postgres-1  Waiting
 Container k8s-hack-postgres-1  Healthy
 Container k8s-hack-api-1  Starting
 Container k8s-hack-api-1  Started
```

That "Waiting" line is `depends_on: condition: service_healthy` actually doing something, not just documentation – `api` doesn't start until `postgres`'s own `healthcheck` (`pg_isready`) reports healthy, because `api` connects to the database on startup and gains nothing by racing it. Both containers come up healthy, and the API works exactly as it will under Kubernetes later in this book:

```
curl -s http://localhost:8000/api/v1/healthz
```

```
curl -s http://localhost:8000/api/v1/items
```

```
{"status":"ok","database":"connected"}
[{"id":"item1","name":"First Item","value":100},{"id":"item2","name":"Second Item","value":200}]
```

Same image, same app code, same `JSONFormatter` from Chapter 11 – `docker compose logs api` shows the identical structured JSON, down to the same `pod` field the app writes into every log line, just holding a container ID (`f2e4d42dc861`) instead of a Kubernetes pod name, because `app.py` reads that field from `HOSTNAME` and Docker sets `HOSTNAME` to the container ID the same way Kubernetes sets it to the pod name. The logging code doesn't know or care which one it's running under.

What Compose gets right

This is the whole deployment: one file, one command, and a `docker-compose.yml` that a new developer can read top to bottom in under a minute and understand exactly what's going to run. There's no cluster to provision first, no separate image-loading step – Kubernetes will need one later, a `minikube image load` – Compose builds straight from the Dockerfile and runs it on the same Docker daemon, immediately. That's real, and it's why Compose is still the right answer for local development even on a project that deploys to Kubernetes in production: the fastest path from "clone the repo" to "the app is running and I can poke at it" almost never runs through a cluster.

Where the ceiling actually is

Push on it a little and the ceiling stops being theoretical. Ask Compose for three copies of `api` instead of one:

```
docker compose up -d --scale api=3
```

```
 Container k8s-hack-api-3  Starting
Error response from daemon: failed to set up container networking: driver failed programming external
connectivity on endpoint k8s-hack-api-3 (80c154af11f7...): Bind for 0.0.0.0:8000 failed: port is already
allocated
```

```
docker compose ps -a
```

```
NAME                  STATUS                     PORTS
k8s-hack-api-1        Up 19 seconds (healthy)    0.0.0.0:8000->8000/tcp
k8s-hack-api-2        Created
k8s-hack-api-3        Created
k8s-hack-postgres-1   Up 25 seconds (healthy)
```

`api-2` and `api-3` exist, as containers, and go no further – `Created`, never `Up`. `docker-compose.yml` maps `api`'s port with `"8000:8000"`, a literal host port, and a host only has one port 8000. The first container to claim it wins; everything after that fails to bind. This isn't a bug or a missing flag. `docker-compose.yml`'s port mapping is written as "this container's port 8000 goes on this host's port 8000," and that sentence has no meaning once there's more than one container trying to be the answer to it. Chapter 5's Service object exists specifically to answer a different question – "route to whichever of these pods is healthy right now" – and that question doesn't have a sensible answer inside a single `docker-compose.yml` file at all, because Compose has no concept of "a stable address in front of more than one container." Multi-host scheduling has the same shape of problem one level up: Compose has one Docker daemon to talk to, on one host, so "which of N machines should this container run on" isn't a question Compose is even positioned to ask.

Rolling updates hit a version of the same wall. `docker compose up --build` after changing `app.py` stops the old `api` container and starts a new one – not simultaneously, not with the old one kept alive until the new one proves itself healthy. Chapter 8 will show Kubernetes rejecting a bad `ConfigMap` change this same way, without ever taking `toy-api` down. Compose's healthcheck exists and works, as `postgres`'s did above, but nothing in Compose reads it to decide whether it's safe to remove an old container yet. That gating logic is exactly what a Deployment's rolling update adds on top of the same healthcheck idea.

The right tool until it isn't

None of this makes `docker-compose.yml` worse than the seven YAML files from Chapter 9's comparison – it's 41 lines against 201, and for a single host running two containers, it is a strictly better fit. The honest way to hold both facts at once: Compose describes an application on one machine, completely and well, and every one of the gaps above – one host, a fixed port that can't be shared, no gating on rollouts – is a gap that only matters once there's

more than one machine, more than one copy of a service, or an update that has to happen without taking anything down. Chapter 9 will come back to this exact tradeoff once there's a full Kubernetes deployment to compare it against squarely. For now, the ceiling is the point: everything Compose can't do in this chapter is a preview of what the next several chapters exist to fix.

Chapter 5

What Kubernetes Actually Is

The control loop, not the orchestrator

Chapter 4 ended at Compose's ceiling: one host, and nothing watching over it once `docker compose up` returns. That second part is the real gap. `docker-compose.yml` describes what should run, `docker compose up` makes it run, and then the description's job is finished. If a container dies five minutes later, nothing goes back and checks the file again. You find out because the app is down, not because anything noticed the file said otherwise.

Kubernetes' actual job is to keep noticing. Not once, at apply time, but continuously, forever, until you change your mind. `deployment.yaml` doesn't say "start 2 containers" – Chapter 6 covers this file in detail, but the shape of it matters here first – it says `replicas: 2`, a fact about how the world ought to look. Something is always comparing that fact against how the world actually looks, and closing the gap whenever the two disagree:

```
loop forever:
    desired = read the spec
    actual  = observe the cluster
    if desired != actual:
        act to close the gap
```

That's the entire idea. Everything else in Kubernetes – the scheduler placing pods, the kubelet keeping containers running, the whole apparatus Chapter 7 puts through its paces – is one more instance of this same loop, watching a different piece of the world. It's why deleting a pod on purpose and a node crashing and killing that pod by accident produce the identical outcome: the loop doesn't know or care why `actual` stopped matching `desired`, only that it did. A Compose restart policy is a rule about one specific failure you anticipated. A control loop has no list of anticipated failures – it just keeps rechecking, so anything that knocks `actual` out of line with `desired` gets corrected the same way, whether you predicted it or not.

The control plane is not magic – it's pods

Ask this cluster what's actually running its control plane:

```
kubectl get pods -n kube-system
```

```
NAME                               READY   STATUS    RESTARTS      AGE
coredns-7d764666f9-t5p4b           1/1     Running   0             25h
etcd-minikube                      1/1     Running   0             25h
kube-apiserver-minikube            1/1     Running   0             25h
kube-controller-manager-minikube   1/1     Running   0             25h
kube-proxy-dpl4j                   1/1     Running   0             25h
kube-scheduler-minikube            1/1     Running   0             25h
storage-provisioner                1/1     Running   2 (16h ago)   25h
```

`kube-apiserver-minikube`, `kube-scheduler-minikube`, `kube-controller-manager-minikube`, `etcd-minikube` – the four pieces usually drawn as a special box labeled "control plane" in every Kubernetes diagram – are just pods, in this same `kubectl get pods` output, next to `coredns` and a storage provisioner. Minikube runs them as ordinary containers because that's what they are. A managed cluster (EKS, GKE, AKS) hides these behind the cloud provider so you're not responsible for keeping etcd alive, but hiding them doesn't change what they are underneath

– four programs, each with one job, watching each other's output the same way `toy-api`'s pods get watched by the Deployment controller.

What each one actually does, briefly:

- **`kube-apiserver`** is the only thing that talks to `etcd` directly. Every other piece here, including `kubectl` itself, only ever talks to the API server. `kubectl apply -f deployment.yaml` is a write to the API server, nothing more.
- **`etcd`** is where the desired state actually lives – not the YAML file on disk, which is only a copy, but this key-value store. The file is how you tell etcd what to remember.
- **`kube-scheduler`** watches for pods that exist in the desired state but haven't been assigned to a node yet, and picks one.
- **`kube-controller-manager`** runs the control loops themselves – the one that keeps a Deployment's replica count correct is one of many bundled in here.

Down on the node, outside this control-plane list, two more pieces close the loop: the **kubelet** watches the API server for pods assigned to its own node and makes sure their containers are actually running, and **kube-proxy** (also visible above, as `kube-proxy-dpl4j`) sets up the networking rules that make a Service's stable address actually route to the right pods. Nothing here is a black box. It's the same watch-diff-act loop from the last section, six times, each instance responsible for one slice of "does reality match the spec."

Mapping what you already know

Chapter 4 walked through this repo's `docker-compose.yml` – two services, `postgres` and `api`, each described by roughly a dozen lines. Every one of those lines has a direct Kubernetes equivalent; it's just split across more files, because Kubernetes gives each concern its own object instead of one file per application:

Compose concept	Kubernetes equivalent
`services.api`	Deployment (`deployment.yaml`)
`services.postgres`	StatefulSet (`postgres-statefulset.yaml`) – Chapter 6 covers why a database gets a different object than a stateless service
`ports: "8000:8000"`	Service (`service.yaml`) – a stable address, decoupled from any one container
`environment:` (non-secret values)	ConfigMap (`postgres-configmap.yaml`)
`environment:` (`POSTGRES_PASSWORD`)	Secret (`postgres-secret.yaml`)
`volumes: postgres_data:...`	PersistentVolumeClaim (`postgres-pvc.yaml`)
`healthcheck:`	`livenessProbe` / `readinessProbe` on the pod spec
`depends_on: condition: service_healthy`	no direct equivalent – Chapter 6 covers how `start.sh` handles this by waiting on readiness explicitly

The table looks like Kubernetes just renamed things and split them into more files, and at the level of "what fields exist," that's not wrong. The actual difference is everything from the first two sections of this chapter: Compose's version of this table describes a single `docker compose up` invocation, done once. Kubernetes' version describes a standing order that a handful of control loops keep enforcing against whatever's actually running, on whichever of however many nodes happen to be available, for as long as the cluster exists. Same information, different verb tense – Compose says "do this," Kubernetes says "keep this true."

Chapter 6

Your First Deployment

`./start.sh`: containers + orchestration, not magic

The shell script `start.sh` establishes the prerequisites you'll need for a small local Kubernetes setup with Minikube. Simply running this script and watching the messages it produces is illuminating. Useful, important stuff is happening, but there is nothing incomprehensible going on.

Deployments, Services, and the separation of "what runs" from "how it's reached"

Open `deployment.yaml` and `service.yaml` side by side. They're two different objects because they answer two different questions.

```yaml
# deployment.yaml, abbreviated
kind: Deployment
spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: api
          image: k8s-toy-api:local
          ports:
            - containerPort: 8000
          env:
            - name: DATABASE_URL
            ... etc
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 500m
              memory: 256Mi
```

`deployment.yaml` answers "what should be running." It says: run 2 copies of the `k8s-toy-api:local` image, give each one a container port of 8000, give it these environment variables, and check `/api/v1/healthz` periodically to decide if it's alive and ready. That's it. Nothing in this file knows or cares how anything outside the pod finds these containers. A Deployment's job ends at "keep this many copies of this container running and healthy."

```yaml
# service.yaml, abbreviated
kind: Service
spec:
  type: NodePort
  selector:
    app: toy-api
  ports:
```

```
    - port: 8000
      targetPort: 8000
      name: http
```

`service.yaml` answers "how do I reach it." The `toy-api` Service doesn't run any code – it's a stable address and a routing rule. It has a `selector: app: toy-api`, meaning it forwards traffic to whatever pods currently carry that label, whatever their names or IPs happen to be at the moment. Pods come and go – they get deleted and recreated with new names and new IPs constantly, that's normal – but the Service's name and address don't change. Anything that wants to talk to the toy API talks to the Service, never to a specific pod.

This split matters because the two things change on different schedules. Scaling the Deployment from 2 replicas to 5, or rolling out a new image, happens often and shouldn't require touching how the app is addressed. Meanwhile the Service's job – routing to whatever's currently healthy – has to keep working through all of that churn without being told about it directly. Separating "what runs" from "how it's reached" is what makes that possible.

The `toy-api` Service in this repo is `type: NodePort`, which opens a port on the Minikube node itself (something in the 30000-32767 range, allocated automatically) so you can hit the API from outside the cluster without port-forwarding. `postgres-service.yaml` uses `type: ClusterIP` instead, with `clusterIP: None` – that's a headless Service, and it exists for a different reason covered in the next section.

One more thing worth flagging in `deployment.yaml`: it sets `imagePullPolicy: IfNotPresent` against the tag `k8s-toy-api:local`, with a comment that the image is built locally and never pulled from a registry. That combination – a mutable tag plus "don't repull if you already have something under this name" – is a known trap: rebuild the image, run `minikube image load` again, and it can silently no-op if a running pod is still holding the old image under that same tag. The fix is to scale the Deployment to zero, let the old pods actually go away, then reload the image and scale back up. Real deployments avoid the whole problem with immutable, content-addressed tags – a git SHA or build digest – so the tag itself is proof of what's running, not just a label that might be stale.

StatefulSets, and why databases aren't just "a Deployment with a volume"

The toy API is a Deployment. Postgres is a StatefulSet. They look almost identical in the YAML – same containers, same probes, same resource limits – so it's worth being specific about why they're not interchangeable.

A Deployment's pods are disposable and identical. If you're running 2 replicas of `toy-api` and one gets killed, Kubernetes starts a replacement with a new name and a new IP, and nothing about the app cares which one you happen to be talking to at a given moment – that's exactly what the Service in the previous section is for. The replicas don't have individual identities worth preserving.

Postgres can't work that way. It has exactly one replica here (`replicas: 1` in `postgres-statefulset.yaml`), and it's bound to one specific `PersistentVolumeClaim` – `postgres-pvc`, defined separately in `postgres-pvc.yaml`. A StatefulSet gives that one pod a stable name (`postgres-0`) and reattaches the same volume to it every time it's recreated. Delete `postgres-0` and Kubernetes brings back `postgres-0` again, not some interchangeable replacement – same name, same storage, same data.

```
# postgres-pvc.yaml, abbreviated
kind: PersistentVolumeClaim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi

# postgres-service.yaml, abbreviated
kind: Service
spec:
  ports:
    - port: 5432
      targetPort: 5432
      protocol: TCP
      name: postgres
  clusterIP: None
```

```yaml
# postgres-statefulset.yaml, abbreviated
kind: StatefulSet
spec:
  serviceName: postgres
  replicas: 1
  template:
    spec:
      containers:
        - name: postgres
          image: postgres:16-alpine
          ports:
            - containerPort: 5432
              name: postgres
          volumeMounts:
            - name: postgres-storage
              mountPath: /var/lib/postgresql/data
      volumes:
        - name: postgres-storage
          persistentVolumeClaim:
            claimName: postgres-pvc
```

That's what "set" means here. It's easy to hear "StatefulSet" and picture something structurally different from a Deployment. But it's the same idea, a controller managing a group of pods from one template. The difference is that the pods aren't just "N copies of this template," they're ordinal slots: `postgres-0`, and if this were scaled to 3 replicas, `postgres-1` and `postgres-2` alongside it, each with its own PersistentVolumeClaim and created and torn down in order, 0 before 1 before 2. A Deployment's pods get random suffixes because their identity doesn't matter. A StatefulSet's pods are numbered because the number *is* the identity. It's how the same pod keeps finding its way back to the same volume every time it's recreated. This repo only ever runs one replica, so you only ever see `postgres-0`, but the indexing is there waiting for the day you need `postgres-1`.

That's also why `postgres-service.yaml` is headless (`clusterIP: None`). A normal Service load-balances across whatever pods match its selector, but that won't work for a database you need to address specifically rather than "any of the following." A headless Service instead gives each StatefulSet pod its own stable DNS name, so other things in the cluster can find `postgres-0` by name if they need to, not just "postgres, whichever one answers."

Stateless replicas can be identical because there's nothing to lose by throwing one away and starting fresh. A database can't be treated that way. Its whole value is the data sitting on disk, tied to one specific process. The StatefulSet exists to preserve exactly that identity: this pod, this volume, every time.

Hands-on: deploying the toy API + Postgres from this repo

`start.sh` runs all of the above, in the order it has to happen. Worth reading top to bottom once, because the order isn't arbitrary:

1. `docker build -t k8s-toy-api:local .` – build the image
2. `minikube image load k8s-toy-api:local` – get that image into Minikube's own image store, since Minikube runs its own Docker daemon separate from your host's
3. Delete anything already deployed under these filenames, ignoring errors if nothing's there yet
4. Apply the ConfigMap and Secret first, then the PVC, then the Postgres StatefulSet and Service, then the API's Deployment and Service – config before the things that consume it, storage before the thing that claims it, database before the app that connects to it
5. `kubectl wait --for=condition=ready pod -l app=postgres --timeout=120s` – block here until Postgres actually reports ready via its `readinessProbe`, not just "created"
6. Same wait for the API pods
7. Print `kubectl get pods,svc,pvc` so you can see everything that exists
8. Run `test-api.sh` – a full CRUD pass against the live API, over its NodePort or `localhost:8000` depending on what's running
9. Print instructions for reaching the API yourself afterward

Run it and read the output as it happens rather than skipping to the end. The `Waiting for PostgreSQL to be ready` line means what it says – until Postgres's own `pg_isready` check passes, nothing else is allowed to proceed, because the API pods talk to Postgres on startup and gain nothing by starting before there's a database to talk to. When it works, the whole thing looks almost boring: build, load, apply, wait, test, done. That's the point. There's

no step in here that isn't something you could explain to someone else in one sentence.

The YAML files as promises kept

Go back through every file `start.sh` applied, but read them a second time with a different question in mind: not "what does this field do" but "what is Kubernetes promising to keep true, and for how long."

`postgres-configmap.yaml` and `postgres-secret.yaml` are the promise that changing the database name or password never means touching `app.py` or rebuilding an image – config and code are separable, permanently, by construction. `postgres-pvc.yaml` is the promise that the 1Gi of storage it requests will still hold the same data no matter what happens to the pod sitting on top of it. `postgres-statefulset.yaml` is the promise, covered above, that `postgres-0` and its volume stay paired through every reschedule. `postgres-service.yaml`'s headless setup is the promise that something can always find that specific pod by name, not just "postgres, whichever one answers." `deployment.yaml`'s probes are the promise that a container which stops answering gets pulled out of rotation and replaced without anyone watching for it. `service.yaml` is the promise that the address for reaching the API doesn't move even while the pods behind it are constantly being replaced.

None of this is a new set of facts – it's the same seven files already walked through above. What changes is the lens: every field in every one of these manifests exists because it's holding up a specific guarantee, and once you can name the guarantee, the YAML stops looking like configuration syntax and starts looking like a list of promises with the evidence attached.

It's also worth being honest about where those promises stop. A bare StatefulSet's guarantees are exactly two: stable identity, stable storage. That's all `postgres-statefulset.yaml` is promising. It says nothing about replication, nothing about failover if the node running `postgres-0` dies, nothing about backups. Real production databases close that gap with an Operator sitting on top of the StatefulSet – more on that later.

Chapter 7

Self-Healing and Scaling

Deleting a pod on purpose and watching Kubernetes notice

With the toy API and Postgres both running, list the current pods and delete one of the `toy-api` ones directly:

```
kubectl get pods -l app=toy-api
kubectl delete pod toy-api-6cf667676b-9477m   # use a name from your own output
```

Then watch what happens next:

```
kubectl get pods -l app=toy-api -w
```

A replacement pod appears within a second, in `Running` but not yet `1/1` ready, and reaches `1/1` a few seconds later once its readiness probe passes:

```
NAME                       READY   STATUS    RESTARTS      AGE
toy-api-6cf667676b-fwzzq   0/1     Running   0             1s
toy-api-6cf667676b-n5j7p   1/1     Running   1 (22h ago)   22h
toy-api-6cf667676b-fwzzq   1/1     Running   0             7s
```

The pod you deleted is gone for good – it's a new name, not a resurrection. Confirm you're back to the expected replica count and both pods report ready:

```
kubectl get pods -l app=toy-api
```

Nobody ran a script to make that happen. Nothing in `deployment.yaml` says "if a pod dies, start another one". That instruction doesn't need to exist anywhere. It says `replicas: 2`, and the Deployment controller's whole job is making the cluster's actual pod count match that number, continuously, forever, regardless of why the count drifted. You deleting a pod on purpose and a node crashing and killing a pod by accident look identical from the controller's point of view: the observed state stopped matching the desired state, so it acts.

Declarative intent: a standing order, not a one-time command

This is different from a script triggered by a commit hook or a CI/CD pipeline. A shell script that runs `docker run` twice starts two containers and then it's done. Its job is finished the moment the command returns. `replicas: 2` in a Deployment spec isn't a command that finishes; it's a standing order the control loop keeps re-checking against reality, indefinitely, until someone changes the spec. Nothing "runs" it on a schedule. It's just always being checked.

That's why deleting a pod gets you a replacement but deleting the Deployment itself does not – the standing order is gone, so there's nothing left to re-check against.

Scaling out: easy for the API, meaningless for the database

Scaling the API is a one-line change in intent:

```
kubectl scale deployment/toy-api --replicas=4
```

Four pods, each an independent copy of the same stateless process, each capable of answering any request identically, because none of them are holding onto anything the others don't have. The Service's label selector picks up the two new pods automatically – nothing about the Service definition changes.

Trying the equivalent on Postgres exposes exactly why "stateful scaling" isn't a smaller version of the same problem:

```
kubectl scale statefulset/postgres --replicas=2
```

`postgres-1` comes up, `1/1` Ready, looking exactly as healthy as `postgres-0`. But check what it's actually connected to:

```
kubectl exec postgres-1 -- psql -U postgres -d toyapi -c "SELECT * FROM items;"
```

```
 id | name | value
----+------+-------
(0 rows)
```

Empty. `postgres-1` isn't a replica of `postgres-0` with a copy of its data – it's a second, completely independent Postgres process that happens to be running the same image. Prove it by inserting a row into `postgres-1` and checking whether `postgres-0` ever sees it:

```
kubectl exec postgres-1 -- psql -U postgres -d toyapi -c \
  "INSERT INTO items (id, name, value) VALUES ('probe', 'from postgres-1', 777);"
kubectl exec postgres-0 -- psql -U postgres -d toyapi -c \
  "SELECT * FROM items WHERE id='probe';"
```

The insert succeeds against `postgres-1`. The `SELECT` against `postgres-0` comes back empty: `(0 rows)`. They don't know about each other. Nothing in a bare StatefulSet spec says "and also replicate the data between instances," because a StatefulSet's job stops at stable identity and stable storage, the same two guarantees from the previous section, and not one guarantee more. Getting actual replication requires either an application that knows how to replicate itself (Postgres does, but it has to be configured to) or an Operator that automates that configuration. `replicas: 2` on a StatefulSet gets you two databases, not one database with a backup.

Scale back down and confirm `postgres-0`'s data was never touched in the first place:

```
kubectl scale statefulset/postgres --replicas=1
kubectl wait --for=condition=ready pod -l app=postgres --timeout=60s
kubectl exec postgres-0 -- psql -U postgres -d toyapi -c "SELECT * FROM items;"
```

```
  id   |    name     | value
-------+-------------+-------
 item1 | First Item  |   100
 item2 | Second Item |   200
```

Same two rows as before the experiment started. `probe` never existed as far as `postgres-0` is concerned, because it never did – it was inserted into a different process's disk entirely, and that process is gone now that `postgres-1` has been scaled away.

Load balancing across replicas, observed via logs

Back on the API side, scale up again and send a batch of requests at the Service, hitting its NodePort directly rather than any one pod:

```
kubectl scale deployment/toy-api --replicas=4
kubectl wait --for=condition=ready pod -l app=toy-api --timeout=60s

NODE_PORT=$(kubectl get svc toy-api -o jsonpath='{.spec.ports[0].nodePort}')
MINIKUBE_IP=$(minikube ip)
for i in $(seq 1 20); do
  curl -s -o /dev/null "http://${MINIKUBE_IP}:${NODE_PORT}/api/v1/items"
done
```

Then check each pod's own logs for how many of those twenty requests it answered, using the `"items listed"` line that `list_items` logs on every call:

```
for pod in $(kubectl get pods -l app=toy-api -o jsonpath='{.items[*].metadata.name}'); do
  count=$(kubectl logs "$pod" --since=60s | grep -c '"items listed"')
  echo "$pod: $count requests"
done
```

```
toy-api-6cf667676b-4bv8l: 6 requests
toy-api-6cf667676b-fwzzq: 2 requests
toy-api-6cf667676b-j867p: 6 requests
toy-api-6cf667676b-n5j7p: 6 requests
```

Every pod got some traffic, but not an even split. `kube-proxy` is choosing a backend at random for each connection, not round-robining – so with only twenty requests, some unevenness is expected, and it would smooth out over a few thousand. The behavior worth internalizing isn't "perfectly balanced" – it's "any of these four processes can answer, and the caller never had to know or care which one did."

Chapter 8

Configuration, Secrets, and Storage

Editing a ConfigMap live, and finding the edge of "hot reload"

`postgres-config` holds the non-secret pieces of the database connection – host, port, database name, username – and `deployment.yaml` wires each one into the API container as an environment variable via `configMapKeyRef`. Patch the ConfigMap while the app is running:

```
kubectl patch configmap postgres-config --type merge \
  -p '{"data":{"POSTGRES_USER":"postgres_renamed"}}'
kubectl get configmap postgres-config -o jsonpath='{.data.POSTGRES_USER}'
```

```
postgres_renamed
```

The ConfigMap object shows the new value immediately. But check the same variable inside an already-running pod:

```
POD=$(kubectl get pods -l app=toy-api -o jsonpath='{.items[0].metadata.name}')
kubectl exec "$POD" -- env | grep POSTGRES_USER
```

```
POSTGRES_USER=postgres
```

Still the old value. This is the edge of what a ConfigMap actually promises: Kubernetes updates the object right away, but a container's environment variables are set once, at container start, from whatever the ConfigMap said at that moment – nothing pushes changes into a running process's environment afterward. `env` isn't a live view of the ConfigMap; it's a snapshot taken once.

Getting the new value into the app means restarting the pods that read it:

```
kubectl rollout restart deployment/toy-api
```

Except in this case the new value is `postgres_renamed`, and Postgres was never told that role exists – `POSTGRES_USER` only sets up Postgres's own initial role at first-ever startup, and `postgres-0` had already been initialized long before this edit. Watch it happen in real time:

```
kubectl get pods -l app=toy-api -w
```

```
NAME                       READY   STATUS             RESTARTS
toy-api-6cf667676b-fwzzq   1/1     Running            0
toy-api-6cf667676b-n5j7p   1/1     Running            1 (22h ago)
toy-api-76c48f9b5c-5t9q6   0/1     CrashLoopBackOff   3 (17s ago)
```

Check why the new pod is unhappy:

```
kubectl logs toy-api-76c48f9b5c-5t9q6 --tail=5   # use your own new pod's name
```

```
asyncpg.exceptions.InvalidPasswordError: password authentication failed for user "postgres_renamed"
ERROR:    Application startup failed. Exiting.
```

Both old pods stay `1/1 Running` the entire time, and the Service keeps answering `200` on every health check throughout:

```
NODE_PORT=$(kubectl get svc toy-api -o jsonpath='{.spec.ports[0].nodePort}')
MINIKUBE_IP=$(minikube ip)
curl -s -o /dev/null -w "%{http_code}\n" "http://${MINIKUBE_IP}:${NODE_PORT}/api/v1/healthz"
```

```
200
```

Nothing about the API went down. That's `RollingUpdate` doing exactly what it's specified to do (`maxUnavailable: 25%`, `maxSurge: 25%` by default): it won't remove an old, healthy pod until a new one proves itself ready, and a pod stuck crash-looping never gets there. A bad config change here fails safely – loudly, in `kubectl get pods`, but without taking anything down.

Revert and confirm the rollout completes cleanly this time:

```
kubectl patch configmap postgres-config --type merge \
  -p '{"data":{"POSTGRES_USER":"postgres"}}'
kubectl rollout restart deployment/toy-api
kubectl rollout status deployment/toy-api --timeout=60s
```

```
deployment "toy-api" successfully rolled out
```

Both pods have new names now – the crash-looping one is gone, and so is the previously-healthy old pod it never managed to replace, cleaned up in the same rollout once a working replacement finally passed its readiness probe.

PersistentVolumeClaims: storage that outlives the pod

Chapter 6 covered why `postgres-pvc.yaml` exists – the promise that data survives independent of any particular pod. Worth actually watching that promise get kept:

```
kubectl exec postgres-0 -- psql -U postgres -d toyapi -c "SELECT * FROM items;"
```

```
  id   |    name     | value
-------+-------------+-------
 item1 | First Item  |   100
 item2 | Second Item |   200
```

Now delete the pod and wait for its replacement:

```
kubectl delete pod postgres-0
kubectl wait --for=condition=ready pod -l app=postgres --timeout=60s
kubectl exec postgres-0 -- psql -U postgres -d toyapi -c "SELECT * FROM items;"
```

```
  id   |    name     | value
-------+-------------+-------
 item1 | First Item  |   100
 item2 | Second Item |   200
```

Same rows, before and after. The pod that answers the second query isn't the same process as the one that answered the first – it was deleted and recreated in between – but the StatefulSet reattached the same `postgres-pvc` volume to the new `postgres-0`, so from the data's perspective nothing happened. This is the actual mechanism behind "storage outlives the pod": not magic persistence, just the same PVC getting claimed again by whatever process the StatefulSet starts under that name next.

ConfigMap and Secret, side by side

`postgres-configmap.yaml` holds four fields – host, port, database name, username. `postgres-secret.yaml` holds exactly one – the password. Every one of those five values ends up in the same place: `deployment.yaml` reads all of them into environment variables on the `api` container, and one more variable, `DATABASE_URL`, is built by string-substituting all five together.

```yaml
# deployment.yaml, abbreviated
env:
  - name: POSTGRES_HOST
    valueFrom:
      configMapKeyRef:
        name: postgres-config
        key: POSTGRES_HOST
  # ...POSTGRES_PORT, POSTGRES_DB, POSTGRES_USER the same way
  - name: POSTGRES_PASSWORD
    valueFrom:
      secretKeyRef:
        name: postgres-secret
```

```
        key: POSTGRES_PASSWORD
  - name: DATABASE_URL
    value:
postgresql://$(POSTGRES_USER):$(POSTGRES_PASSWORD)@$(POSTGRES_HOST):$(POSTGRES_PORT)/$(POSTGRES_DB)
```

Same `env` list, same `valueFrom` mechanism, same moment at container start when the value gets read in. If you only looked at timing, a ConfigMap and a Secret are identical – which is exactly what the earlier experiment already proved for the ConfigMap half: edit it, and a running pod keeps the old value until something restarts it. A Secret behaves no differently on that score. So the split isn't about *when* Kubernetes hands the value to the container. It's about what kind of thing the value is once it's sitting in etcd or showing up in `kubectl` output.

Ask the cluster for the ConfigMap and it just tells you:

```
kubectl get configmap postgres-config -o yaml
```

```
apiVersion: v1
data:
  POSTGRES_DB: toyapi
  POSTGRES_HOST: postgres
  POSTGRES_PORT: "5432"
  POSTGRES_USER: postgres
kind: ConfigMap
```

Ask for the Secret the same way:

```
kubectl get secret postgres-secret -o yaml
```

```
apiVersion: v1
data:
  POSTGRES_PASSWORD: cG9zdGdyZXM=
kind: Secret
type: Opaque
```

`cG9zdGdyZXM=` is not encrypted – it's base64, which anyone can decode in one command (`echo cG9zdGdyZXM= | base64 -d` gets you back `postgres`). Kubernetes isn't hiding the password from someone with access to read Secrets in this namespace. What the Secret type actually buys you is narrower and easy to undersell: it's excluded from `kubectl describe`'s normal output, it can be RBAC-scoped separately from ConfigMaps so "who can read config" and "who can read passwords" are different permissions, and it signals to every tool in the ecosystem – Sealed Secrets, External Secrets, Vault integrations – that this value belongs to a different handling path than everything else in the pod's environment. `describe pod` makes the distinction visible in one place:

```
kubectl describe pod toy-api-6fc5ddfd6d-7gn64   # use your own pod's name
```

```
Environment:
  POSTGRES_HOST:      <set to the key 'POSTGRES_HOST' of config map 'postgres-config'>  Optional: false
  POSTGRES_PORT:      <set to the key 'POSTGRES_PORT' of config map 'postgres-config'>  Optional: false
  POSTGRES_DB:        <set to the key 'POSTGRES_DB' of config map 'postgres-config'>    Optional: false
  POSTGRES_USER:      <set to the key 'POSTGRES_USER' of config map 'postgres-config'>  Optional: false
  POSTGRES_PASSWORD:  <set to the key 'POSTGRES_PASSWORD' in secret 'postgres-secret'>  Optional: false
  DATABASE_URL:
  postgresql://$(POSTGRES_USER):$(POSTGRES_PASSWORD)@$(POSTGRES_HOST):$(POSTGRES_PORT)/$(POSTGRES_DB)
```

Four lines name their ConfigMap and show nothing else to hide. The fifth names its Secret and stops there – `kubectl describe` won't print a Secret's value even if you're staring right at the pod that consumes it. Same list, same mechanism, one visibly different rule for one line, and that line is the one line in this file that was never meant to be readable in passing.

`postgres-secret.yaml` says the quiet part out loud in a comment: `stringData` is used here "for readability," and in production you'd reach for Sealed Secrets or External Secrets instead. Both of those close the actual gap – neither etcd nor a `kubectl get -o yaml` from someone with namespace access should be enough to read a real password – but that's a problem for a later chapter. What this repo demonstrates is the shape of the split, not yet the hardened version of it.

Chapter 9

When Not to Use Kubernetes

The same app, two ways

This repo has both versions sitting side by side. `docker-compose.yml` is 41 lines and two services:

```yaml
services:
  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: toyapi
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 3s
      retries: 5

  api:
    build:
      context: .
      dockerfile: Dockerfile
    environment:
      DATABASE_URL: "postgresql://postgres:postgres@postgres:5432/toyapi"
    depends_on:
      postgres:
        condition: service_healthy
```

`docker compose up` and both containers are running, `api` waiting for `postgres`'s healthcheck before it starts, both reachable on their mapped ports. That's the whole deployment.

The Kubernetes version of the identical app – same image, same Postgres, same environment variables – is seven YAML files totaling 201 lines (`postgres-configmap.yaml`, `postgres-secret.yaml`, `postgres-pvc.yaml`, `postgres-statefulset.yaml`, `postgres-service.yaml`, `deployment.yaml`, `service.yaml`), plus `start.sh`, 63 lines of shell to apply them in the right order and wait for each one to actually come up before moving on to the next. That's not a criticism of either file – both are doing what they were designed to do, correctly. It's the actual, measured cost of the thing Chapter 5 called a control loop: seven times more YAML, plus an orchestration script Compose doesn't need at all, to run the same two containers on the same one machine.

What all that extra machinery is buying, here, right now

Go back through the "why Kubernetes" list from Chapter 5's control-loop framing – multi-host scheduling, self-healing across machines, rolling updates with real health gating – and check which of those this repo's minikube setup actually has:

- **Multi-host scheduling?** No. Minikube is one node. Every pod in this repo, `toy-api` and `postgres` alike, lands on the same machine, every time.

- **Survives a node dying?** No, not meaningfully – there's only the one node. If it goes down, everything on it goes down, StatefulSet or not.
- **Multiple teams sharing a cluster?** No. It's one person running one app on one laptop.
- **Autoscaling on real load?** Not configured, and there's no load to scale against.

What's actually being exercised is the Deployment controller replacing a pod you killed on purpose (Chapter 7), and a rolling update refusing to take down a healthy pod for a bad config change (Chapter 8). Both real, both worth learning – and both things `docker compose up --scale` and a restart policy get you a version of, on one box, without seven YAML files. The honest reading of this repo's own setup is that it's demonstrating Kubernetes mechanics on a workload that doesn't yet need Kubernetes. That was the point – it's a learning exercise, not a counterargument to Chapter 5. But it's also exactly the shape of the mistake `WHY_KUBERNETES.md` warns about: reaching for the fleet-management tool before there's a fleet.

What running Kubernetes well actually requires

The 201 lines of YAML are the part you write once. The part that doesn't show up in any file is what it costs to run this well past a learning cluster: someone has to own etcd's health, because etcd is the one thing in this whole system that has no fallback if it loses quorum. Someone has to keep the control plane's Kubernetes version current, because managed control planes still expect you to handle node-side upgrades. Someone has to actually read `kubectl get pods` output and notice a `CrashLoopBackOff` before a customer does, because Kubernetes will happily leave a broken Deployment in that state forever without paging anyone on its own. `WHY_KUBERNETES.md` puts the comparison plainly: a managed database (RDS, Cloud SQL) makes the cloud provider your Operator; a bare StatefulSet like `postgres-statefulset.yaml` in this repo makes *you* the Operator, and an Operator's job is a lot more than the two guarantees – stable identity, stable storage – that a StatefulSet actually provides. That's a team's worth of ongoing attention, not a weekend project, and it's a cost that exists whether or not you're using any of the capacity it buys you.

Signs you don't need it yet, and signs you're about to

The `docker-compose.yml` in this repo is the honest baseline: one host, one team, a healthcheck and a restart policy cover the failure modes that actually happen. Stay there as long as it keeps being true. The signals that it's stopped being true tend to show up as specific, concrete events, not vague unease – one host running out of headroom under real load, a deploy that needs to happen without an outage because people are actively using the thing, a second team that needs to ship independently without stepping on the first team's containers, or a hardware failure that actually took something down and everyone found out that "restart policy" doesn't cover "the whole box is gone." Any one of those is a specific problem Compose has no answer for and Kubernetes was built around. Reaching for Kubernetes before any of them has actually happened buys you 201 lines of YAML and an operational burden with nothing yet to show for it – which is exactly what this repo's minikube setup is, deliberately, as a place to learn the mechanics before you need them for real.

Chapter 10

Infrastructure as Code, Take One: Pulumi

Same resources, different syntax

Everything in Chapters 6 through 8 came from YAML files applied with `kubectl apply -f`. `pulumi/__main__.py` in this repo does the same kind of thing – describe a ConfigMap, a Deployment, a Service, hand it to Kubernetes – but as Python instead of YAML:

```
config_map = k8s.core.v1.ConfigMap(
    "graph-api-config",
    metadata=k8s.meta.v1.ObjectMetaArgs(name="graph-api-config"),
    data={
        "UVICORN_LOG_LEVEL": "info",
        "UVICORN_PORT": "8000",
    },
)
```

Read past the class names and this is the same shape as `postgres-configmap.yaml`: a name, a `data` dict. The difference isn't what's being described, it's what's doing the describing – a real programming language instead of a document format. That distinction sounds abstract until you hit the places where YAML has no good answer and Python does.

One example already sitting in this file: the Service is `NodePort` with no `node_port` pinned, so Kubernetes allocates one from the 30000-32767 range at apply time – you don't know it in advance. `start.sh`'s answer to that, back in Chapter 6, was a separate shell command run after the fact: `kubectl get svc toy-api -o jsonpath=...`. Pulumi's answer is to make the allocated port part of the program itself:

```
node = k8s.core.v1.Node.get("cluster-node", node_name)

node_ip = node.status.apply(
    lambda s: next(a.address for a in s.addresses if a.type == "InternalIP")
)
node_port = service.spec.apply(lambda s: str(s.ports[0].node_port))

pulumi.export("base_url", pulumi.Output.concat("http://", node_ip, ":", node_port))
```

`node.status`, `service.spec` – these aren't plain Python values, they're `Outputs`, Pulumi's name for "a value that doesn't exist yet because the resource that produces it hasn't been created yet." `.apply()` is how you say "once this exists, do this with it." The comment right above the export line in the actual file explains why that distinction can't be skipped: `pulumi.Output.concat` is used deliberately instead of an f-string, because an f-string would silently interpolate the Python `repr()` of the `Output` object itself rather than the eventual string value. That's not a hypothetical gotcha – it's specific enough that whoever wrote this file had clearly been bitten by it once. A YAML manifest can't have this problem, because YAML has no concept of "a value that will exist later." It also can't hand you a working URL without a second, separate shell command to go find the allocated port – the same tradeoff in both directions.

Type checking and reusable modules

`k8s.core.v1.ConfigMap(...)`, `k8s.apps.v1.DeploymentSpecArgs(...)` – every one of these is a real Python class with a real constructor signature. Misspell a field name in a YAML manifest and you find out at `kubectl apply` time, if you're lucky, or at pod-crash time if you're not. Misspell one of these and your editor tells you before you run anything, because `DeploymentSpecArgs` doesn't have a `replicaz` parameter and static analysis knows it. That's what "type checking" is actually buying here – not a vague quality signal, a concrete class of typo that never reaches a cluster.

The same file demonstrates reuse too, if you look at how the Prometheus and Grafana blocks lower down are built. They don't share a function with the `graph-api` Deployment above them, but they share the same pattern – `labels` dict, `Deployment`, `Service`, wired together the same way – and in Python that repetition is a standing invitation to extract a `make_deployment(name, image, port, ...)` helper the moment a fourth service shows up. A YAML manifest has no equivalent move. You can use Helm templates or Kustomize overlays to fight the same duplication, but that's reaching for a second tool to patch a gap in the first one. Pulumi just uses the language you're already in.

Where this file actually stands right now

This Pulumi program is not a clean parallel of the toy-api YAML from Chapters 6 through 8. It deploys an image called `tg-core-graph-api:local`, from a different exercise (`tg-core`) than the `k8s-toy-api:local` this book has been walking through, and along the way it's picked up a Prometheus deployment scraping `graph-api`'s `/metrics` endpoint and a Grafana deployment wired to that Prometheus as its one datasource. None of that was in the file when this repo's `README.md` was written – the README still describes it as creating "the exact same ConfigMap + Deployment + Service as the YAML manifests," which was true once and isn't anymore.

This is now a small, harmless instance of a problem, but it's one that gets expensive at real scale: documentation describes the infrastructure as of whenever someone last updated the doc, and the infrastructure-as-code keeps moving. A YAML manifest and a Pulumi program are both supposed to be the source of truth for what's running – that's the entire pitch of this part of the book – but nothing enforces that a README stays truthful about either one. The fix isn't clever tooling, it's the same discipline Chapter 2 argued for at the start: *if a description of the system lives outside the system's own declared state, it drifts, and the only real defense is noticing.*

When Pulumi earns its complexity over plain manifests

None of this makes Pulumi strictly better than YAML. It's more machinery: a language runtime, a package manager, a state backend that has to be reachable and correctly authenticated before `pulumi up` does anything at all. The Pulumi program in this repo is configured against a remote backend, and if that backend is unreachable, `pulumi stack ls` fails outright before you get anywhere near applying anything. `kubectl apply -f service.yaml` has no equivalent failure mode; the manifest is the state.

This extra machinery becomes worthwhile when a project has more than a handful of resources, when the same shapes (a Deployment plus a Service plus a ConfigMap, over and over) start repeating across services, or when "I made a typo in a field name" has actually cost someone a debugging session. That's when a real language starts paying for itself. A single toy API with one ConfigMap doesn't need it. This repo's own Pulumi file, three services deep and still growing, is starting to sit right at that line.

Chapter 11

Observability Basics: Logging

One line becomes three, and a pod name you didn't ask for

Hit the running API once, with a request ID attached so it's easy to pick back out of the noise:

```
curl -s -H "X-Request-ID: book-demo-0001" \
  "http://${MINIKUBE_IP}:${NODE_PORT}/api/v1/items"
```

Then go looking for it in the logs of both `toy-api` pods:

```
for pod in $(kubectl get pods -l app=toy-api -o jsonpath='{.items[*].metadata.name}'); do
  echo "=== $pod ==="
  kubectl logs "$pod" --since=30s | grep book-demo-0001
done
```

```
=== toy-api-6fc5ddfd6d-7gn64 ===
=== toy-api-6fc5ddfd6d-b9kvv ===
{"timestamp": "2026-09-07T18:47:23.462567Z", "level": "info", "message": "request started", "request_id":
"book-demo-0001", "pod": "toy-api-6fc5ddfd6d-b9kvv", "method": "GET", "path": "/api/v1/items", "client":
"10.244.0.1"}
{"timestamp": "2026-09-07T18:47:23.469232Z", "level": "info", "message": "items listed", "request_id":
"book-demo-0001", "pod": "toy-api-6fc5ddfd6d-b9kvv", "count": 2}
{"timestamp": "2026-09-07T18:47:23.471122Z", "level": "info", "message": "request completed", "request_id":
"book-demo-0001", "pod": "toy-api-6fc5ddfd6d-b9kvv", "method": "GET", "path": "/api/v1/items", "status":
200}
```

One `curl`, three log lines, and nothing came out of `toy-api-...-7gn64` at all – the request landed on `b9kvv` because that's whichever pod `kube-proxy` happened to route it to, same randomness from Chapter 7. Nobody chose that pod on purpose and the request didn't care which one answered, but once you're debugging instead of just calling the API, you suddenly do care, and `request_id` is the only thing that ties the three lines together as one event rather than three unrelated ones.

That correlation isn't automatic. Open `app.py` and it's exactly two pieces of machinery: a `ContextVar` holding the current request's ID, and `RequestIDMiddleware`, which reads `X-Request-ID` off the incoming request (or generates a UUID if the caller didn't send one), sets it in that ContextVar, and echoes it back in the response headers. Every `logger.info(...)` call downstream, in every endpoint, picks it up for free through `JSONFormatter`, because the formatter reads the same ContextVar on every single line it emits:

```
request_id = request_id_ctx.get()
if request_id:
    log_data["request_id"] = request_id
```

That's the whole trick. No log line anywhere in `app.py` passes `request_id` explicitly – `logger.info("items listed", extra={"count": len(items)})` in the items endpoint has no idea what request it's part of. The formatter fills that in from context, on every line, unconditionally. Miss wiring that ContextVar into some new code path – a background task, a second thread – and its log lines quietly stop carrying a `request_id` at all, with nothing to warn you.

The line the endpoint wrote vs. the line the formatter added

Try a request that fails on purpose:

```
curl -s -o /dev/null -w "%{http_code}\n" \
  -H "X-Request-ID: book-demo-404" \
  "http://${MINIKUBE_IP}:${NODE_PORT}/api/v1/items/does-not-exist"
```

```
404
```

```
{"timestamp": "2026-09-07T18:47:33.879184Z", "level": "info", "message": "request started", "request_id":
"book-demo-404", "method": "GET", "path": "/api/v1/items/does-not-exist", "client": "10.244.0.1"}
{"timestamp": "2026-09-07T18:47:33.881006Z", "level": "warning", "message": "item not found", "request_id":
"book-demo-404", "item_id": "does-not-exist"}
{"timestamp": "2026-09-07T18:47:33.881569Z", "level": "info", "message": "request completed", "request_id":
"book-demo-404", "method": "GET", "path": "/api/v1/items/does-not-exist", "status": 404}
```

The middle line is the one that actually says something – `logger.warning("item not found", extra={"item_id": item_id})`, written by hand in the `get_item` endpoint, one call, one string, one extra field. Everything else on that line – `timestamp`, `level`, `service`, `pod`, `request_id` – came from `JSONFormatter` without the endpoint author having to think about any of it. That split is the actual point of structured logging: the application code stays down at "here's what happened" (`item not found`, `does-not-exist`), and the formatter's job is making sure every line, no matter which of the dozen or so `logger.info`/`logger.warning` calls scattered through `app.py` produced it, comes out shaped the same way and carries the same request-level context. `grep`-ing plain text for an error means guessing at a string. Piping this through `jq 'select(.level=="warning")'` means asking a question the log already knows the answer to.

Not every field on that line is one you'd choose on purpose, though. Look closely at the full JSON as it actually comes out of the pod – there's a `taskName` field in there too, something like `"taskName": "starlette.middleware.base.BaseHTTPMiddleware.__call__.<locals>.call_next.<locals>.coro"`. That's `JSONFormatter` doing exactly what it's written to do – copying every attribute off the `LogRecord` that isn't on its exclusion list – and asyncio happens to stash the current task's name on the record under a key nobody thought to exclude. It's harmless here, just noise, but it's a fair warning about the "log everything on the record" approach: the formatter doesn't know the difference between a field you meant to add and one the runtime left lying around.

Where the trail actually ends

Send one more request, note which pod answers, and delete that pod on purpose:

```
curl -s -o /dev/null -H "X-Request-ID: book-demo-vanish" \
  "http://${MINIKUBE_IP}:${NODE_PORT}/api/v1/items"
kubectl logs toy-api-6fc5ddfd6d-7gn64 --since=30s | grep book-demo-vanish
```

```
{"timestamp": "2026-09-07T18:47:43.309449Z", "level": "info", "message": "request started", "request_id":
"book-demo-vanish", "pod": "toy-api-6fc5ddfd6d-7gn64", ...}
{"timestamp": "2026-09-07T18:47:43.312656Z", "level": "info", "message": "items listed", "request_id":
"book-demo-vanish", "pod": "toy-api-6fc5ddfd6d-7gn64", "count": 2}
{"timestamp": "2026-09-07T18:47:43.313184Z", "level": "info", "message": "request completed", "request_id":
"book-demo-vanish", "pod": "toy-api-6fc5ddfd6d-7gn64", ...}
```

The line's there, plainly, with the pod's name right on it. Now delete that pod the same way Chapter 7 did, wait for its replacement, and ask for the same logs again:

```
kubectl delete pod toy-api-6fc5ddfd6d-7gn64
kubectl wait --for=condition=ready pod -l app=toy-api --timeout=60s
kubectl logs toy-api-6fc5ddfd6d-7gn64
```

```
error: error from server (NotFound): pods "toy-api-6fc5ddfd6d-7gn64" not found in namespace "default"
```

Gone. Not "gone from the default view, still there if you dig" – gone. `kubectl logs` reads from the node, keyed on a pod that no longer exists, and the Deployment controller's whole job, going all the way back to Chapter 7, is making sure that pod's replacement gets a new name. The same mechanism that makes Kubernetes self-healing is what makes `kubectl logs` alone useless for anything you need to look back on. A crash worth debugging is exactly the kind of event likely to have killed the pod that logged it.

`LOGGING.md` calls this "node-level logging" and lists Loki or the EFK stack as the fix: ship every line off the node to something with its own retention, before the pod that wrote it disappears. Nothing about that requires changing a single line in `app.py`. The JSON is already there, already structured, already carrying `request_id` and `pod` on every

line – Promtail or Fluentd's whole job is reading what's already being written to stdout and shipping it somewhere that outlives the pod. The logging code in this repo was written for that day already; today it's just not running yet.

Chapter 12

Authentication and Authorization Patterns

Two questions that sound like one

"Who can do that" is actually two unrelated questions once there's a Kubernetes cluster involved, and this repo has a clean answer to one of them and no answer at all to the other. The first question: who can call `toy-api`'s endpoints – list items, create one, delete one. The second: who can run `kubectl apply`, `kubectl delete pod`, or anything else against the cluster itself. Nothing about answering one tells you anything about the other. A person with full `kubectl` access to this cluster can't necessarily call a protected endpoint on `toy-api` if the app checks its own credentials separately, and a service with a valid API key for `toy-api` has no Kubernetes permissions at all unless someone explicitly granted them. They're enforced by different code, at different layers, and mixing them up is exactly the confusion this chapter exists to clear up.

The application side: an API key, actually wired in

Before adding anything, check what an anonymous request to `toy-api` could do:

```
curl -s -o /dev/null -w "%{http_code}\n" -X DELETE \
  "http://${MINIKUBE_IP}:${NODE_PORT}/api/v1/items/item1"
```

```
200
```

That's not a hypothetical – it deleted `item1`, no credentials of any kind. `AUTH.md` lays out real options for closing this gap: API keys for service-to-service calls, JWTs for anything with a notion of a user, OAuth2/OIDC for handing that off to an external identity provider entirely, mTLS at the service-mesh layer. API keys are the simplest of the four, which makes them the right one to actually wire in here – `app.py` gets one new dependency function, applied only to the three endpoints that change data:

```
async def require_api_key(x_api_key: Annotated[str | None, Header()] = None) -> None:
    """Gate write endpoints behind a shared API key."""
    if settings.api_key is None:
        raise HTTPException(status_code=503, detail="API key not configured")
    if x_api_key is None or not secrets.compare_digest(x_api_key, settings.api_key):
        raise HTTPException(status_code=401, detail="Missing or invalid API key")

@app.post("/items", dependencies=[Depends(require_api_key)])
@app.put("/items/{item_id}", dependencies=[Depends(require_api_key)])
@app.delete("/items/{item_id}", dependencies=[Depends(require_api_key)])
```

`secrets.compare_digest` instead of `==` matters here for the same reason it matters anywhere credentials get compared: a naive `==` short-circuits on the first mismatched character, which makes the comparison's timing leak how many leading characters were right. It's a narrow attack against a toy API on a laptop, and a real one against anything exposed to the internet. `list_items` and `get_item` get no dependency at all – reads stay open, same as `/api/v1/healthz` has to stay open regardless of which auth scheme gets picked, since Kubernetes' liveness and readiness probes call it directly with no mechanism for presenting credentials.

The key itself follows the exact pattern `postgres-secret.yaml` set up in Chapter 8 – a new `api-key-secret.yaml`, wired into `deployment.yaml` as one more `secretKeyRef`:

```
kubectl describe pod toy-api-5fd8ff4d68-c7vdn | grep -A 7 "Environment:"
```

```
Environment:
  POSTGRES_HOST:      <set to the key 'POSTGRES_HOST' of config map 'postgres-config'>  Optional: false
  ...
  API_KEY:            <set to the key 'API_KEY' in secret 'api-key-secret'>  Optional: false
```

Same masking, same mechanism, one more line. Now the same anonymous request:

```
curl -s -w "\nstatus: %{http_code}\n" -X DELETE \
  "http://${MINIKUBE_IP}:${NODE_PORT}/api/v1/items/item2"
```

```
{"detail":"Missing or invalid API key"}
status: 401
```

Reads still don't need anything:

```
curl -s "http://${MINIKUBE_IP}:${NODE_PORT}/api/v1/items"
```

```
[{"id":"item1","name":"First Item","value":100},{"id":"item2","name":"Second Item","value":200}]
```

And the correct key gets through:

```
curl -s -w "\nstatus: %{http_code}\n" -X DELETE \
  "http://${MINIKUBE_IP}:${NODE_PORT}/api/v1/items/item2" \
  -H "X-API-Key: demo-key-a1b2c3d4e5"
```

```
{"status":"deleted","id":"item2"}
status: 200
```

That's the whole feature, and it's also close to the ceiling of what API keys are good for. There's one key, shared by every legitimate caller – `test-api.sh` uses the same one a real client would. Revoking access for one caller without affecting the others means rotating the key for everyone, because the key doesn't identify *who's* calling, only *that* they know the secret. `AUTH.md`'s JWT section is the fix for that – a token that carries an identity and an expiry, not just a shared password – and OAuth2/OIDC is the fix for not wanting to issue or verify tokens yourself at all. Both are real upgrades over what's here now, and neither was needed to demonstrate the actual point of this section: an unauthenticated write and an authenticated one are now provably different requests, not the same request either way.

The cluster side: RBAC, checked directly

Cluster RBAC is a separate system, answering a separate question, and this repo's manifests never touch it – no `Role`, no `RoleBinding`, no `ServiceAccount` of its own. Check what identity the `toy-api` pods actually run under:

```
kubectl get pods -l app=toy-api -o jsonpath='{.items[0].spec.serviceAccountName}'
```

```
default
```

Every pod in the `default` namespace that doesn't specify a `serviceAccountName` gets this one, automatically, whether anyone thought about it or not. Ask Kubernetes directly what that identity is allowed to do against the API server – not what `toy-api`'s own code permits, what the *cluster* permits this ServiceAccount to touch:

```
kubectl auth can-i list pods --as=system:serviceaccount:default:default
kubectl auth can-i get secrets --as=system:serviceaccount:default:default
kubectl auth can-i delete deployments --as=system:serviceaccount:default:default
```

```
no
no
no
```

Three flat no's. The `default` ServiceAccount, absent any `Role` granting it something, can't list pods, can't read a Secret – including `postgres-secret`, sitting right there in the same namespace – can't delete a Deployment. Compare that to whatever identity `kubectl` itself is using, the one this whole book has been running commands as:

```
kubectl auth can-i --list
```

```
Resources   Non-Resource URLs   Resource Names   Verbs
*.*         []                  []               [*]
```

`*.*` and `[*]` – every resource, every verb. That's minikube's own admin credential, configured into `~/.kube/config` when the cluster started, and it's a completely different identity from the one `toy-api`'s pods run under. One has unrestricted access to everything in the cluster. The other has none. Both are true at the same moment, about the same cluster, because `kubectl auth can-i` and `curl`-ing an endpoint on `toy-api` are checking two unrelated permission systems that happen to share the word "auth."

On AWS: the same ServiceAccount does more work

This repo targets minikube because minikube is free and local, not because minikube is the destination – the eventual home for something like `toy-api` is a real cluster, and EKS is AWS's version of that. The `default` ServiceAccount's job changes the moment `toy-api` makes that move, not because Kubernetes RBAC works any differently, but because AWS adds a second question on top of it: not just "what can this identity do to the Kubernetes API," but "what can this identity do to AWS itself." A pod that needs to read from S3, publish to SQS, or call any other AWS service needs AWS credentials, and handing every node's EC2 instance role broad permissions – so that every pod scheduled on that node inherits them, whether it needs them or not – was the original, blunt answer. Tools like `kube2iam` and `kiam` existed specifically to patch that blunt answer, intercepting each pod's request for credentials and handing back something narrower. AWS's own answer, **IRSA** (IAM Roles for Service Accounts), replaces that interception trick with something built into the platform.

The mechanism is the same `ServiceAccount` object this chapter has already been checking with `kubectl auth can-i` – IRSA just adds one annotation:

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: toy-api
  namespace: default
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::111122223333:role/toy-api-role
```

That annotation is a pointer to an IAM role, and the IAM role's own trust policy is what makes the binding specific rather than a free-for-all: it names the cluster's OIDC provider as a trusted federated identity, and restricts the trust to one exact `system:serviceaccount:<namespace>:<name>` string – this ServiceAccount, in this namespace, nothing broader. A mutating admission webhook, watching for pods that use an annotated ServiceAccount, injects a projected token and two environment variables (`AWS_ROLE_ARN`, `AWS_WEB_IDENTITY_TOKEN_FILE`) into the pod at creation time. The AWS SDK picks those up automatically and exchanges the token for short-lived AWS credentials scoped to exactly that IAM role – the pod never sees a long-lived access key, and a pod using a *different* ServiceAccount gets none of it. Same shape as everything else in this chapter: an identity, a scope, a binding that has to be created on purpose or the default is nothing.

AWS introduced a newer mechanism in late 2023, **EKS Pod Identity**, that does the same job with less setup – no OIDC provider to register, a DaemonSet running on each node instead of a per-cluster trust-policy dance, and an IAM role's trust policy naming a fixed AWS service principal instead of a cluster-specific OIDC ARN, so the same role can be reused across clusters without editing it. It's not a deprecation of IRSA – Pod Identity doesn't support Fargate, Windows nodes, or EKS Anywhere, so IRSA is still the only option there – but it's the simpler default worth reaching for on a standard Linux-EC2 EKS cluster.

Both of those answer "what can a pod do to AWS." The reverse question – who's allowed to `kubectl apply` against an EKS cluster at all, the same question `kubectl auth can-i --list` answered for minikube above – is a separate mechanism again: an `aws-auth` ConfigMap mapping IAM users/roles to Kubernetes RBAC groups, or, since December 2023, EKS **access entries**, which do the same mapping through the EKS API instead of a ConfigMap anyone with `edit` on `kube-system` could otherwise patch. Three related but distinct systems, then, once AWS is in the picture – Kubernetes RBAC for what's allowed inside the cluster, IRSA or Pod Identity for what a pod is allowed to do to AWS, and access entries for who's allowed to reach the cluster's API at all – and the pattern repeats: nothing is granted by default, every binding is a specific, auditable choice, and the `default` ServiceAccount having nothing at all is still, on AWS as much as on minikube, what "nobody asked for anything" correctly looks like.

Where each one actually belongs

The practical rule falls out of what got demonstrated above rather than needing to be stated as a separate principle: application auth belongs inside the application, checked in `app.py`, because only the application knows what "delete an item" or "read this user's data" actually means. Cluster RBAC belongs to whatever is allowed to change the

cluster's own state – a CI pipeline's deploy credentials, a human running `kubectl` by hand, the ServiceAccount a controller uses to watch and reconcile other objects. `toy-api` itself has no legitimate reason to ever call the Kubernetes API, so the `default` ServiceAccount having zero permissions isn't a gap to close, it's the correct state by accident – the app simply never asked for more, and nothing granted it any.

The one place these two systems are supposed to meet, deliberately, is a controller or operator – something Chapter 6 flagged as the gap a bare StatefulSet leaves open. A Postgres Operator, unlike `toy-api`, has a real reason to talk to the Kubernetes API: creating PVCs, updating StatefulSets, watching for pod failures. That's exactly the case where a `ServiceAccount` needs a real `Role` scoped to what the operator actually does – read and write StatefulSets and PVCs in its own namespace, nothing about Secrets in other namespaces, nothing cluster-wide. Getting that scope right is most of what makes an Operator trustworthy to run: too little and it can't do its job, too much and a bug or a compromise in the operator's code becomes a cluster-wide problem instead of a contained one. `toy-api`'s `default` ServiceAccount, with nothing granted, is what "too little" looks like when nothing was ever asked for. A real operator sits somewhere in between, and getting that middle right is a RoleBinding written on purpose, not a default nobody thought about.

Chapter 13

Git as the Control Plane

The inversion

Every deployment so far in this book has been a push. `kubectl apply -f deployment.yaml` from Chapter 6 onward means: something outside the cluster – a person, a script – has credentials for the cluster and sends it a command. The cluster is passive. It waits to be told.

GitOps inverts that. Instead of something outside the cluster pushing changes in, something *inside* the cluster watches a git repository and pulls changes toward itself, continuously, on its own schedule. Nobody outside the cluster needs credentials for the cluster at all – they need credentials for git, which is a fundamentally smaller thing to leak or misuse. `deployment.yaml` doesn't change. What changes is who's holding the "apply" button, and it turns out the answer "a controller running inside the cluster, watching git" is both more secure and more auditable than "whoever currently has `kubectl` access."

More auditable because every change is now a commit – Chapter 2's whole argument, version control as a security control, showing up again here as the mechanism rather than the abstract principle. More secure because push credentials are the more dangerous kind: a leaked `kubectl` config is a direct line into the cluster, while a leaked git read token gets an attacker a copy of some YAML, not a shell. The worst a compromised git credential can do is see what's already supposed to be public inside the team, or, if it's a write credential, propose a change that still has to pass through commit history and, ideally, review – not silently reach into the cluster and start deleting things.

ArgoCD's loop is the same loop

This isn't a new idea bolted onto Kubernetes. It's Chapter 5's control loop, run one level up:

```
loop forever:
    desired = read the git repo
    actual  = observe the cluster
    if desired != actual:
        act to close the gap
```

Compare that to the loop Chapter 5 wrote for the Deployment controller – identical shape, different source for `desired`. The Deployment controller reads its spec from etcd; ArgoCD reads its spec from a git remote. Both are controllers, in the exact sense Chapter 5 used the word: something that watches, diffs, and acts, running inside the cluster, indefinitely, without anyone re-triggering it. ArgoCD isn't a CI/CD tool bolted onto Kubernetes from outside – it's a Kubernetes-native controller whose one job is polling an external system (git) instead of watching another API object, then reconciling exactly the way every other controller in this book already does.

Hands-on: this repo, deployed by ArgoCD, for real

This book has a second cluster running alongside the minikube one from every earlier chapter – a `kind` cluster, with ArgoCD installed, and a Gitea instance holding a mirror of this repo. `argocd-k8s-hack.yaml`, sitting at the root of this repo, is the ArgoCD `Application` object that ties them together:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
```

```yaml
metadata:
  name: k8s-hack
  namespace: argocd
spec:
  project: default
  source:
    repoURL: http://172.22.0.3:3000/wware/k8s-hack.git
    targetRevision: main
    path: .
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated:
      prune: false
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

`repoURL` is Gitea's address on the `kind` network – `172.22.0.3`, not `localhost`, because ArgoCD is asking for that address from inside a pod, and `localhost` from inside a pod means the pod itself. `targetRevision: main` is the whole of "what ArgoCD should watch." Everything after that is ArgoCD's job, not this repo's.

Prove it by actually moving git and watching the cluster follow. `main` currently has `replicas: 2` in `deployment.yaml`. Change it, commit, push:

```
git checkout main
# edit deployment.yaml: replicas: 2 -> replicas: 3
git commit -am "Scale toy-api to 3 replicas"
git push gitea main
```

ArgoCD polls every three minutes by default; force it to look now instead of waiting, the same way a person clicking "Refresh" in the UI would:

```
kubectl patch application k8s-hack -n argocd --type merge \
  -p '{"metadata":{"annotations":{"argocd.argoproj.io/refresh":"hard"}}}'
```

```
kubectl get pods -n default -l app=toy-api
```

```
NAME                       READY   STATUS    RESTARTS   AGE
toy-api-8467757567-kglph   1/1     Running   0          19s
toy-api-8467757567-kx8t6   1/1     Running   0          18d
toy-api-8467757567-wpsph   1/1     Running   0          18d
```

A third pod, 19 seconds old, next to two that have been running for eighteen days. Nobody ran `kubectl apply`. Nobody but ArgoCD touched this cluster – the only thing that changed was a file in a git repository, and a controller running inside the cluster noticed and closed the gap on its own.

What "the cluster is passive" actually buys

Push that a step further. Manually scale the same Deployment directly, bypassing git entirely, the way an incident responder under pressure might:

```
kubectl scale deployment/toy-api -n default --replicas=5
```

```
kubectl get pods -n default -l app=toy-api
```

```
NAME                       READY   STATUS        RESTARTS   AGE
toy-api-8467757567-97htw   0/1     Terminating   0          3s
toy-api-8467757567-wpsph   1/1     Running       0          18d
toy-api-8467757567-w4kb9   0/1     Terminating   0          3s
toy-api-8467757567-kglph   1/1     Running       0          40s
toy-api-8467757567-kx8t6   1/1     Running       0          18d
```

Two of the five pods this command just started are already `Terminating` before they even finish coming up. This Application has `selfHeal: true` set – the syncPolicy block from `argocd-k8s-hack.yaml` above – so ArgoCD isn't waiting for anyone to notice the drift and click Sync. It's actively defending git's declared state against exactly the kind of manual change that just happened, correcting it faster than the new pods could finish starting:

```
kubectl get pods -n default -l app=toy-api

NAME                         READY   STATUS    RESTARTS   AGE
toy-api-8467757567-kglph   1/1     Running   0          66s
toy-api-8467757567-kx8t6   1/1     Running   0          18d
toy-api-8467757567-wpsph   1/1     Running   0          18d
```

Back to three, matching git, within seconds of the manual scale command completing. `kubectl apply` from Chapter 5 through Chapter 12 always won – it was the only thing touching the cluster's desired state. Here it loses, immediately, to whatever git says, because git is the actual source of truth and `kubectl scale` was never anything more than a temporary, local lie about it. That's the practical payoff of the inversion this chapter opened with: the cluster no longer trusts whoever's holding a terminal. It trusts a git remote, continuously, whether or not anyone's watching.

Chapter 14

Drift Detection and Self-Healing at the Fleet Level

Two settings, two very different outcomes

`selfHeal: true` was doing a lot of quiet work in the last chapter's demo – the drift got corrected so fast that "cluster disagrees with git" and "cluster back in sync with git" were nearly the same moment. That's the right behavior for production, and it's also easy to mistake for the whole story. Flip `selfHeal` off and the same manual scale command produces something genuinely different: not a faster or slower correction, but no correction at all, until someone decides there should be one.

```
kubectl patch application k8s-hack -n argocd --type merge \
  -p '{"spec":{"syncPolicy":{"automated":{"prune":false,"selfHeal":false}}}}'

kubectl scale deployment/toy-api -n default --replicas=5

kubectl get pods -n default -l app=toy-api

NAME                       READY   STATUS    RESTARTS   AGE
toy-api-8467757567-czvzk   1/1     Running   0          8s
toy-api-8467757567-kglph   1/1     Running   0          5m17s
toy-api-8467757567-kx8t6   1/1     Running   0          18d
toy-api-8467757567-qjhtd   1/1     Running   0          8s
toy-api-8467757567-wpsph   1/1     Running   0          18d
```

Five pods, and nothing takes them away. Compare that to Chapter 13's `selfHeal: true` demo, where two of five pods were already terminating before they'd finished starting. With `selfHeal` off, the same manual scale just… works, and keeps working, indefinitely, exactly as if ArgoCD weren't involved at all.

The diff is there, even when nothing acts on it

That's the part worth sitting with – "nothing corrects it" is not the same as "nothing notices it." Force a refresh and ask ArgoCD what it thinks:

```
kubectl get application k8s-hack -n argocd

NAME       SYNC STATUS   HEALTH STATUS
k8s-hack   OutOfSync     Healthy
```

Two separate judgments, and it's worth being precise about the difference, because conflating them is an easy mistake. `HEALTH STATUS: Healthy` is Kubernetes' opinion of `toy-api` right now: five pods, all of them `Running`, all of them passing their readiness probes – nothing about that state looks unwell to a Deployment controller or to ArgoCD's own health checks. `SYNC STATUS: OutOfSync` is a completely different question: does the cluster's actual state match what git says it should be. The app is healthy *and* wrong at the same time, and both of those are true, permanently, until someone or something acts. Ask for detail and ArgoCD names the exact resource that disagrees:

```
kubectl get application k8s-hack -n argocd \
  -o jsonpath='{.status.resources}' | python3 -m json.tool | grep -B3 OutOfSync
```

```
"kind": "Deployment",
"name": "toy-api",
"status": "OutOfSync",
```

Nothing else in the app – not the Service, not the ConfigMap, not Postgres – shows up in that list, because nothing else changed. The diff is scoped exactly to what actually drifted, which is what makes it usable: a real incident produces a real, specific diff, not a vague "something's off" alert.

A permanent diff, not a one-time audit

This is the distinction the chapter title is pointing at. A traditional audit is something that runs once, or on a schedule – someone, or some script, compares production against what it's supposed to be, produces a report, and that report is stale the moment it's generated. `OutOfSync` isn't a report. It's a live property of the Application object, computed continuously, visible in the same `kubectl get application` command a minute from now, an hour from now, or next week, for exactly as long as the drift persists. Nobody has to remember to go check. The check is always already running.

With `selfHeal: false`, closing the gap is a deliberate act – the equivalent of clicking Sync in the ArgoCD UI, or, from the terminal, triggering the same operation directly:

```
kubectl patch application k8s-hack -n argocd --type merge -p '{
  "operation": {"sync": {"revision": "HEAD", "prune": false}}
}'

kubectl get pods -n default -l app=toy-api

NAME                         READY   STATUS    RESTARTS   AGE
toy-api-8467757567-kglph   1/1     Running   0          6m1s
toy-api-8467757567-kx8t6   1/1     Running   0          18d
toy-api-8467757567-wpsph   1/1     Running   0          18d
```

Back to three, `SYNC STATUS` back to `Synced` – but only because something explicitly said so. That's the honest tradeoff `selfHeal: false` is making: drift gets surfaced immediately and reliably, and reconciling it is a choice, not an automatic reflex. For a system where someone wants eyes on every correction before it happens – a compliance requirement, a genuinely fragile piece of infrastructure – that choice is the point, not a limitation.

Flip `selfHeal` back on and the same drift resolves itself, no `Sync` command needed:

```
kubectl patch application k8s-hack -n argocd --type merge \
  -p '{"spec":{"syncPolicy":{"automated":{"prune":false,"selfHeal":true}}}}'
kubectl scale deployment/toy-api -n default --replicas=1

kubectl get pods -n default -l app=toy-api

NAME                         READY   STATUS    RESTARTS   AGE
toy-api-8467757567-kx8t6   1/1     Running   0          18d
toy-api-8467757567-sd2r5   0/1     Running   0          5s
toy-api-8467757567-z5bsj   0/1     Running   0          5s
```

Two replacement pods already coming up, seconds after a scale-down to one, with nobody touching `Sync` at all.

Even the control file can drift

This chapter's own demo turned up a real instance of the thing it's about, unplanned. `argocd-k8s-hack.yaml` – the file in this repo that git is supposed to treat as the source of truth for the Application itself – says `selfHeal: false`. The live Application, the whole time this chapter's commands were running against it, actually had `selfHeal: true`. Someone flipped it live, at some point, the same way `kubectl scale` flips a replica count live, and never pushed the matching change back to the file. That's drift too – not in what `toy-api` runs, but in the GitOps configuration that's supposed to be governing it, and it's exactly as invisible as any other drift until someone thinks to check. The fix is the same fix as everywhere else in this book: make the file say what's actually true.

```
# argocd-k8s-hack.yaml
    automated:
      prune: false
      selfHeal: true
```

There's a small irony worth naming plainly: the tool built to keep everything else honest against git had, itself, quietly stopped being honest against its own git-tracked spec. GitOps reconciles what ArgoCD is told to manage. It doesn't reconcile ArgoCD's own configuration against itself – that boundary, and where responsibility for watching it actually sits, is worth remembering the next time something that's "supposed to be self-healing" turns out not to be.

Chapter 15

Multi-Environment Deployment Without the Copy-Paste

Three environments, one template

Everything so far in Part IV has been one Application watching one path in one repo. The `gitops-lab-envs` ApplicationSet sitting in the same cluster is a different shape entirely – one generator, scanning a directory, producing as many Applications as it finds matching subdirectories:

```
spec:
  generators:
    - git:
        repoURL: http://172.22.0.3:3000/wware/gitops-lab.git
        revision: main
        directories:
          - path: envs/*
  template:
    metadata:
      name: gitops-lab-{{path.basename}}
    spec:
      destination:
        namespace: '{{path.basename}}'
      source:
        path: '{{path}}'
```

`envs/*` matches `envs/dev`, `envs/staging`, `envs/prod` – three directories in the repo, none of them mentioned by name anywhere in this file. Ask the cluster what actually exists because of it:

```
kubectl get applications -n argocd
```

```
NAME                SYNC STATUS   HEALTH STATUS
gitops-lab-dev      Synced        Healthy
gitops-lab-prod     Synced        Healthy
gitops-lab-staging  Synced        Healthy
```

Three Applications, none of them hand-written. The generator found three directories and rendered the same template three times, substituting `{{path}}` and `{{path.basename}}` differently each time – the same loop-over-a-list mental model from Chapter 13, except the loop variable comes from scanning a directory tree instead of a hardcoded list, and the loop body is a YAML template instead of a function call.

What actually differs between them

The template is identical across all three; what's not identical is what each directory contains. `dev`'s and `prod`'s copies of `deployment.yaml` are the same shape – same `kind`, same container, same selector – and different in exactly the numbers that should differ:

```
# envs/dev/deployment.yaml, abbreviated
spec:
```

```
  replicas: 1
  template:
    spec:
      containers:
        - resources:
            requests: {cpu: 25m, memory: 32Mi}
            limits: {cpu: 100m, memory: 64Mi}
          env:
            - name: ENVIRONMENT
              value: "dev"

# envs/prod/deployment.yaml, abbreviated
spec:
  replicas: 3
  template:
    spec:
      containers:
        - resources:
            requests: {cpu: 100m, memory: 128Mi}
            limits: {cpu: 500m, memory: 256Mi}
          env:
            - name: ENVIRONMENT
              value: "prod"
```

Prod runs three replicas at four times dev's resource requests. That gradient is real, running right now, and checkable directly rather than just asserted from the YAML:

```
kubectl get pods -n dev
kubectl get pods -n staging
kubectl get pods -n prod
```

```
NAME                            READY   STATUS    RESTARTS   AGE
gitops-lab-6dc89657bb-x45vc   1/1     Running   0          18d

NAME                           READY   STATUS    RESTARTS   AGE
gitops-lab-7c8b4d8db-s2224   1/1     Running   0          18d
gitops-lab-7c8b4d8db-x6j4b   1/1     Running   0          18d

NAME                           READY   STATUS    RESTARTS   AGE
gitops-lab-b44b8ff99-cdjj7   1/1     Running   0          18d
gitops-lab-b44b8ff99-mtmsp   1/1     Running   0          18d
gitops-lab-b44b8ff99-txw9d   1/1     Running   0          18d
```

One pod, two pods, three pods – `dev`, `staging`, `prod`, in that order, matching `replicas: 1` / `2` / `3` in each directory's own manifest. Three separate namespaces, three separate Applications, all traceable back to one `ApplicationSet` object and a directory listing.

Why hand-maintained per-environment YAML rots

The alternative to this is the thing every team eventually does by hand: copy `deployment.yaml` into a `staging` folder, copy it again into `prod`, and tweak the numbers in each copy. That works, once. The problem shows up later, when something needs to change everywhere at once – a new environment variable, an updated readiness probe path, a different image tag – and now it needs to be edited in three files that have no enforced relationship to each other beyond having started as copies. Nothing stops `staging/deployment.yaml` from silently missing the same fix `prod/deployment.yaml` got, because nothing ties them together once the copy-paste happens. The drift Chapter 14 spent a whole chapter making visible for cluster-vs-git state is exactly the failure mode three unlinked YAML copies invite between themselves, except there's no ArgoCD watching for *that* kind of drift – nothing flags `staging` and `prod` disagreeing about something they were never supposed to disagree about in the first place.

The ApplicationSet's generator-plus-template split is the actual fix, not a cosmetic one: everything that's supposed to be identical across environments – the `Deployment` kind, the container name, the `CreateNamespace=true` sync option – lives in exactly one place, the template. Everything that's supposed to differ – replica count, resource limits, which namespace it lands in – lives in the one place that's allowed to vary, each environment's own directory. A bug in the shared shape gets fixed once, in the template, and every environment inherits the fix on its next sync. A bug in one environment's specific numbers stays contained to that environment's own file, because that file is the only place those numbers exist. Copy-paste YAML makes both kinds of change error-prone in the same way; splitting

generator from template makes each kind of change exactly as easy as it should be, and no easier than it should be to accidentally miss.

Chapter 16

Autoscaling on Real Signal: KEDA

What CPU can't see

The Horizontal Pod Autoscaler's default signal is CPU utilization, and for a lot of workloads that's a reasonable proxy for "busy." It's a bad proxy for anything that spends most of its time waiting – a worker blocked on I/O, holding a connection open, sitting idle between messages, can be doing genuinely important work while its CPU graph looks flat. A queue-driven worker is close to the worst case for this: it might use almost no CPU while messages pile up behind it, because the bottleneck was never the CPU, it was throughput per worker. Scaling on CPU in that situation means the autoscaler stays quiet exactly when there's a real backlog building, because the metric it's watching was never measuring the thing that actually mattered.

KEDA – Kubernetes Event-Driven Autoscaling – doesn't replace the HPA to fix this. It feeds it something better. Every KEDA `ScaledObject` creates a real, ordinary `HorizontalPodAutoscaler` behind the scenes:

```
kubectl get hpa -n keda-demo
```

```
NAME                          REFERENCE                   TARGETS      MINPODS   MAXPODS   REPLICAS
keda-hpa-queue-worker-scaler   Deployment/queue-worker    0/5 (avg)    1         10        4
```

Same object Kubernetes has always had. What's different is where the `0/5` comes from – not `cpu.usage`, but an `External` metric KEDA publishes itself:

```
kubectl get hpa keda-hpa-queue-worker-scaler -n keda-demo -o yaml
```

```yaml
metrics:
- external:
    metric:
      name: s0-rabbitmq-work-queue
    target:
      averageValue: "5"
      type: AverageValue
  type: External
```

`keda-metrics-apiserver`, one of the three pods KEDA installs, is what makes `s0-rabbitmq-work-queue` a metric the HPA can read at all – it polls RabbitMQ's queue depth and exposes it through the same metrics API the HPA already knows how to consume. KEDA's actual contribution isn't a new autoscaler. It's a new, pluggable source of truth for the one Kubernetes already has.

Watching it scale from zero, for real

`keda-demo/worker.yaml` starts the `queue-worker` Deployment at `replicas: 0` on purpose – there's nothing to consume when the queue is empty, so nothing should be running:

```
kubectl get pods -n keda-demo -l app=queue-worker
```

```
No resources found in keda-demo namespace.
```

Send twenty messages – `keda-demo/send.sh` publishes them onto `work-queue` – and the `ScaledObject`'s trigger, `value: "5"`, does the arithmetic: twenty messages at five per worker is four workers.

```
bash keda-demo/send.sh
```

```
kubectl get deployment queue-worker -n keda-demo
```

```
NAME           READY   UP-TO-DATE   AVAILABLE   AGE
queue-worker   4/4     4            4           17d
```

Four, not a guess – exactly what the trigger's own math predicts, and checkable against the worker logs actually processing distinct tasks in parallel:

```
[pod/queue-worker-...-csv2b] [WORKER] Processing task: task-0
[pod/queue-worker-...-cszw4] [WORKER] Processing task: task-3
[pod/queue-worker-...-wbxtn] [WORKER] Processing task: task-2
[pod/queue-worker-...-wjsk9] [WORKER] Processing task: task-1
```

Four workers, four different tasks in flight at once, each one an independent pod KEDA created because the queue said there was enough backlog to justify it – not because a human ran `kubectl scale`, and not because a CPU graph crossed a threshold that happened to correlate with load.

Scaling back to zero, and paying for the gap honestly

The workers finish, the queue empties, and `cooldownPeriod: 60` in the `ScaledObject` means KEDA waits a full minute of confirmed-empty before it trusts that the burst is really over – worth having, since scaling to zero and immediately back up on the next message would cost more in pod-startup latency than it saves in idle compute. After that minute:

```
kubectl get pods -n keda-demo -l app=queue-worker
```

```
NAME                            READY   STATUS        RESTARTS   AGE
queue-worker-f576b4497-csv2b   1/1     Terminating   0          92s
queue-worker-f576b4497-cszw4   1/1     Terminating   0          89s
queue-worker-f576b4497-wbxtn   1/1     Terminating   0          89s
queue-worker-f576b4497-wjsk9   1/1     Terminating   0          89s
```

And once they're gone:

```
kubectl get hpa -n keda-demo
```

```
NAME                            REFERENCE                   TARGETS               REPLICAS
keda-hpa-queue-worker-scaler   Deployment/queue-worker    <unknown>/5 (avg)     0
```

`<unknown>` instead of `0/5` is worth noticing rather than glossing over – with no pods running, there's no current value to average, and the HPA says so plainly instead of reporting a fake zero. That's a small honest detail in a system built entirely around honest signals: KEDA doesn't pretend to know something it doesn't, the same way it doesn't pretend CPU is measuring something it isn't.

Zero pods means zero cost for exactly as long as there's nothing to do, which is the actual payoff this chapter's title is pointing at. A CPU-based autoscaler with `minReplicas: 1` – the sensible-sounding default, since scaling from zero on CPU alone means nothing is running to generate the CPU signal that would trigger scaling up – pays for at least one idle worker permanently, waiting for load that might not arrive for hours. A queue has no such chicken-and-egg problem: message count is visible whether or not any worker exists to consume it yet, so scaling from zero is not just possible but the natural steady state. Bursty, queue-driven work is common enough in real systems – background jobs, webhook processing, batch pipelines – that "pay for actual work, not idle capacity" isn't a niche optimization. It's what the workload actually looks like, once the autoscaler is watching the right signal.

Chapter 17

The Cost/Complexity Decision

The same problem, three price points

Chapter 16's KEDA demo runs on a `kind` cluster on a laptop, and that's worth being honest about: this book has been demonstrating queue-driven autoscaling for free. A queue-driven worker fleet that scales 0→N on demand is a real, common pattern – and there are at least three substrates that can run it, at three very different costs and three very different operational burdens. None of them is the correct answer by default. Each is correct for a specific situation, and the mistake this chapter is trying to head off is picking one because it's the most familiar or the most impressive, rather than because it matches what's actually being built.

Option A: single-box autoscaling

One machine, no cluster. `docker compose up --scale worker=N`, or a small hand-rolled watcher polling a queue and adjusting replica counts itself, or Nomad in single-node mode if the watcher needs to be more than a weekend project. All three do the real thing – scale workers up when a queue has backlog, down when it doesn't – without any of the machinery Part III and this Part have spent a dozen chapters building.

The honest cost comparison, cribbed from `gitops-lab`'s own decision matrix:

	Single box	Kubernetes (EKS)
Monthly cost	$15-30 (one `t3.medium`)	$210-265 (control plane + nodes)
Setup time	~30 minutes	2-3 hours
Max scale	~20-30 containers, CPU/memory bound	hundreds of nodes
High availability	None – one box, one failure domain	Multi-node, self-healing

For a workload that fits comfortably inside "20-30 containers on one machine" and doesn't need to survive that one machine dying, single-box autoscaling isn't a compromise. It's the option that costs an order of magnitude less and takes a fraction of the setup time, for a workload that was never going to use Kubernetes' actual selling points – Chapter 9's argument again, this time applied specifically to autoscaling rather than to deployment in general.

Option B: AWS Auto Scaling Groups

Skip Kubernetes, keep AWS. An SQS queue depth feeds a `TargetTrackingScaling` policy on an EC2 Auto Scaling Group – structurally the same idea as KEDA's `ScaledObject`, a target metric value per instance instead of per pod, and the ASG launches or terminates EC2 instances to hold that target, the same way KEDA's HPA launches or terminates pods:

```
target_tracking_configuration {
  customized_metric_specification {
    metric_name = "ApproximateNumberOfMessagesVisible"
    namespace   = "AWS/SQS"
  }
```

```
  target_value = 10.0
}
```

Spot instances knock the compute cost down roughly 70% for workloads that can tolerate interruption – batch rendering, data transformation, anything stateless and resumable. What this option doesn't have is KEDA's responsiveness: ASG scaling operates on a 1-3 minute cycle, launching real EC2 instances with real boot times, not scheduling already-warm pods onto already-running nodes. That's a real, structural tradeoff, not a rough edge to be optimized away – an EC2 instance takes minutes to become useful in a way a pod doesn't, no matter how the scaling policy is tuned. Fine for a render farm where a job queued for two extra minutes doesn't matter. Wrong for anything answering requests in real time.

Option C: real EKS

Everything Chapter 16 demonstrated, minus the "on a free laptop cluster" part. A real EKS control plane is a flat $73 a month before a single worker node exists – `gitops-lab`'s own cost breakdown puts a minimally-provisioned cluster with two `t3.medium` nodes, a load balancer, and a NAT gateway at $210-265 a month, running whether or not anything is actually processing work. That number is the honest price of everything Part III and this Part have been walking through for free: self-healing, multi-node scheduling, the whole reconciliation-loop apparatus. None of it is free to run for real, and the fixed costs – control plane, NAT gateway – don't scale down with idle time the way KEDA scales pods down to zero. A cluster costs the same at 2am with nothing in the queue as it does at peak load.

That price is worth paying past a specific threshold: multiple teams sharing infrastructure, workloads that genuinely need multi-node scheduling and not just multiple containers, a scale where "hundreds of nodes" from the table above stops being hypothetical. Below that threshold, EKS is the bulldozer for the garden hole `REAL_EKS_DEPLOY.md` warns about – it works, and it costs $200 a month more than a solution that would have worked just as well.

A decision framework, not a default answer

None of these three options is the right one in general, and that's the actual point. The question worth asking, every time, is the same one Chapter 9 asked about Kubernetes overall: what does this workload actually need, checked against what each option actually costs – not in dollars alone, but in setup time, operational attention, and how much of that cost is fixed versus scales with use. A single box that costs $20 a month and takes thirty minutes to set up is the right answer for far more projects than reach for it. An EKS cluster billing $250 a month whether or not it's doing anything is the right answer only past the point where that fixed cost is smaller than the cost of not having what it buys. Matching the substrate to the actual need, not to what's most familiar or most impressive on a resume, is the whole decision – everything else in this chapter is just making the actual numbers visible enough to make that match honestly.

Chapter 18

Side Quests

EKS emulation on a home LAN

Chapter 17 put a real number on running EKS: $210-265 a month, whether or not anything's actually happening in the cluster. That's a real obstacle to practicing multi-node mechanics before there's a budget or a production reason to spend it – and the obstacle is smaller than it looks, because the part of EKS that's actually EKS-specific is thinner than it seems.

Deployments, Services, RBAC, ConfigMaps, StatefulSets – everything this book has spent seventeen chapters on – is upstream Kubernetes, byte-for-byte identical whether the control plane is a $73/month managed service or `kubeadm` running across a couple of spare machines on a home network. The part that's genuinely AWS-specific is the infrastructure integration layer underneath that API: the VPC CNI handing out real routable IPs, IRSA's OIDC trust dance from Chapter 12, the AWS Load Balancer Controller provisioning a real ALB. That layer doesn't transfer to a home LAN, and pretending it does would defeat the point of practicing.

What does transfer, closely, with real stand-ins rather than fakes: Calico or Cilium for CNI – both are officially supported alternatives on real EKS too, not just a local substitute. MetalLB for `type: LoadBalancer` Services, giving a real LAN IP pool behind the same Service spec that provisions an NLB on EKS – this closes the single most common "my Service just sits `Pending` forever" confusion anyone hits moving off Compose. `local-path-provisioner` for exercising the PVC/StorageClass lifecycle from Chapter 6, even though there's no EBS underneath. None of this is pretend Kubernetes. It's the same control plane, running on hardware instead of a managed service, with the storage and networking layers swapped for honest local equivalents of the same abstractions.

What genuinely doesn't transfer is worth naming rather than glossing over: IRSA needs real IAM, so there's no way to emulate the credential exchange itself locally, only the workload-side pattern – annotating a ServiceAccount, structuring an app to pick up injected credentials – so it's not unfamiliar later. Karpenter and the Cluster Autoscaler can't be practiced without a cloud to scale into. KEDA, though, scales on metrics regardless of what's underneath, which makes Chapter 16's demo the genuinely runnable adjacent skill – multi-node kubeadm plus KEDA covers most of what's operationally different about EKS, without the monthly bill, and the day there's a reason to move, the delta is narrow: swap the CNI config (or keep it, since Calico and Cilium both run on real EKS), swap MetalLB for the AWS Load Balancer Controller, add the IRSA annotations, point at ECR. Everything written above the infrastructure layer – every manifest in this book – doesn't change at all.

Queue-based scaling as a portable pattern

Chapter 16 built one specific instance of a much more general shape: a `ScaledObject` watching a queue, translating depth into replica count. The RabbitMQ trigger this book actually ran:

```
triggers:
- type: rabbitmq
  metadata:
    host: amqp://guest:guest@rabbitmq.keda-demo.svc.cluster.local:5672
    queueName: work-queue
    mode: QueueLength
    value: "5"
```

Put an SQS trigger next to it:

```
triggers:
- type: aws-sqs-queue
  metadata:
    queueURL: https://sqs.us-east-1.amazonaws.com/123456/my-queue
    queueLength: "10"
```

Or Kafka:

```
triggers:
- type: kafka
  metadata:
    bootstrapServers: kafka.kafka.svc.cluster.local:9092
    topic: events
    lagThreshold: "100"
```

Same shape, every time: a `type`, an address for the queue, a threshold that maps to "how much backlog justifies one more worker." The `ScaledObject` around each trigger – `scaleTargetRef`, `minReplicaCount`, `maxReplicaCount` – doesn't change at all between them. Swapping RabbitMQ for SQS in a real system isn't a redesign; it's changing which fifteen lines inside `triggers:` describe the queue, because KEDA's whole job is translating "some external thing has a number that means backlog" into the one thing every `ScaledObject` actually does regardless of where that number came from. The portability isn't an accident of KEDA's API design. It's the same idea Chapter 5 opened this book with, showing up again at this much smaller scale: describe the desired state, let a controller close the gap, and the source of the number driving that description turns out to be one of the least important things about the whole system.

Chapter 19

GitOps Without a Cluster, Watched Live

What Kubernetes was quietly supplying for free

Every reconciliation loop in this book so far – the Deployment controller in Chapter 5, ArgoCD in Chapter 13, KEDA's HPA in Chapter 16 – got to assume something none of them had to build: a live, running system that already knows how to answer "what's actually true right now," continuously, without being asked. That's etcd plus the API server, and it's not a small thing to get for free. Point a controller at a Terraform-managed cloud stack, a Pulumi program, a Docker Compose host, or a Raspberry Pi on someone's LAN, and there's no equivalent already running. Something has to poll git, decide whether the world matches it, and act – from scratch, because the target has no self-diffing control plane sitting underneath it the way Kubernetes always did.

`gitops_reconciler` is a small, real answer to that gap: one wrapper process, driven by a git repo, that can manage several unrelated targets on independent schedules, with the actual apply mechanism – Terraform, Pulumi, Compose, a bare SSH session to a Pi – decided per target. Rather than read about it, run it.

Watching the loop notice a change, live

The demo stack is a small FastAPI app under Docker Compose. Start it the same way `DEMO.md` describes:

```
uv run python reconcile_example.py
```

```
Starting reconciliation for example-app
Compose file: .../example-app/docker-compose.yml
example-app: CHANGED - stack updated successfully
Reconciliation complete
```

`CHANGED`, the first time, because nothing was running yet. Run it again without touching anything:

```
uv run python reconcile_example.py
```

```
example-app: NO_CHANGE - stack already up to date
```

No `docker compose up` this time – the reconciler hashes the rendered compose config and compares it to the hash it saved after the last successful apply, and skips the actual work when nothing changed. Terraform and Pulumi get this kind of diff for free from their own state engines; Compose doesn't have one, so `ComposeBackend` fakes it with a hash comparison instead. Cheap, and honest about being a workaround rather than a real diff.

Now start the watch loop, the actual centerpiece of this chapter:

```
./watch_and_reconcile.sh 8
```

It just calls `reconcile_example.py` every eight seconds, forever – `cron` or a systemd timer in production, a `while true` loop here. Leave it running, and in the repo's working tree, change the compose file's port mapping:

```
sed -i 's/9001:8080/9002:8080/' example-app/docker-compose.yml
git add example-app/docker-compose.yml
git commit -m "Change demo app port to 9002"
```

Nothing was pushed to a remote, and nothing was told to re-run early – the watch loop is already running, on its own eight-second clock, and the next tick finds the change on its own:

```
  [2026-09-09 12:43:23] Running reconciliation...
example-app: NO_CHANGE - stack already up to date

  [2026-09-09 12:43:31] Running reconciliation...
example-app: CHANGED - stack updated successfully
```

One tick still `NO_CHANGE` – the edit hadn't landed yet when that cycle ran – and the very next one, eight seconds later, `CHANGED`. Confirm it's not just a log line:

```
curl -s -o /dev/null -w "9001: %{http_code}\n" http://localhost:9001/
curl -s -o /dev/null -w "9002: %{http_code}\n" http://localhost:9002/
```

```
9001: 000
9002: 200
```

9001 refuses the connection outright – nothing's listening there anymore. 9002 answers. Revert the edit, commit again, and the next tick puts it back exactly the same way, no special-cased "undo" logic anywhere in the reconciler – reverting a git commit and applying a new one are the same operation as far as `apply()` is concerned, which is the same point Chapter 13 made about `git revert` being a real rollback mechanism rather than a separate feature to build.

A worthwhile honest gap, found by accident

Running this demo cold, before making any deliberate change, actually turned up something worth knowing about rather than glossing over: an old `.last_applied_hash` file was still sitting in `example-app/` from an earlier session, and the very first reconciliation reported `NO_CHANGE` even though no containers were running at all. The hash comparison only checks "does the rendered config match what I last successfully applied" – it has no way to notice "and is that still actually running." Delete the containers out from under a `ComposeBackend` by hand, without touching the compose file, and the hash still matches, so the reconciler has no reason to think anything needs fixing. That's a real, narrow gap in hash-based idempotency, not a bug exactly – the hash was never designed to answer "is reality still what I last made it," only "did the desired state change since last time" – and it's worth remembering the next time `NO_CHANGE` shows up somewhere unexpected.

The shared shape underneath all of it

Every backend this reconciler supports – Terraform, Pulumi, CloudFormation, Compose, plain SSH to a Pi – implements the same three methods:

```python
class BackEnd(ABC):
    @abstractmethod
    def apply(self) -> Status: ...

    @abstractmethod
    def destroy(self) -> Status: ...

    @abstractmethod
    def get_outputs(self) -> dict[str, Any]: ...
```

`apply()` reconciles desired state and reports what happened. `destroy()` tears everything down. `get_outputs()` hands back whatever backend-specific data the caller might need – a Terraform output, a Pulumi stack export, a container's IP. That's the entire contract. It's deliberately smaller than it might be – no `plan()`, no dry-run, no built-in approval gate – and Chapter 20 is going to spend real time on why each of those omissions is a choice rather than an oversight. For now, the shape itself is the point: whatever `apply()` was just watched doing to a Docker Compose stack over the last few pages is the exact same method a Terraform-backed or Pulumi-backed target would be running, on its own schedule, against its own git repo, with nothing about the wrapper loop needing to know or care which one it's talking to.

Chapter 20

Design Decisions, and Why They Were Made That Way

Why an ABC instead of a Protocol, checked live

`BackEnd` could have been written as a `typing.Protocol` instead of an `abc.ABC` – structural typing instead of nominal, matching by shape instead of by declared inheritance. Python's own docs would call that the more idiomatic choice for something this small. It's not what `gitops_reconciler` does, and the reason is more concrete than a style preference: `ManagedTarget` is a Pydantic model with a `backend: BackEnd` field, and Pydantic validates that field differently depending on which one it is.

With the real `BackEnd` ABC, hand it something that merely looks like a backend – same three method names, no inheritance:

```
class LooksLikeABackend:
    def apply(self): pass
    def destroy(self): pass
    def get_outputs(self): return {}

ManagedTarget(name="test", backend=LooksLikeABackend(), repo=Path("/tmp"))
```

```
ValidationError: 1 validation error for ManagedTarget
backend
  Input should be an instance of BackEnd [type=is_instance_of, ...]
```

Rejected, immediately, at construction time – before a single `apply()` ever gets called against production. Swap the ABC for a `@runtime_checkable Protocol` and hand it something with the same method *names* but a genuinely wrong signature:

```
class NotReallyABackend:
    def apply(self, target, force=True):
        return "oops, wrong signature entirely"
    def destroy(self): pass
    def get_outputs(self): return {}
```

```
>>> isinstance(NotReallyABackend(), BackEndProtocol)
True
>>> Model(backend=NotReallyABackend())
Model(backend=<...NotReallyABackend object...>)
```

Accepted. `runtime_checkable` only checks that methods with those names exist on the object – not their signatures, not their return types, not whether calling them does anything sane. A `Protocol` is happy to shake hands with something that would blow up the moment the wrapper actually called `apply()` for real. This is the same argument Chapter 10 made about Pulumi's typed classes catching a misspelled field before `pulumi up` ever runs, one layer further down: not "which syntax reads nicer," but "which one of these two choices catches the mistake before it reaches production instead of during it."

Why there's no `plan()` or dry-run method

The obvious instinct, coming from Terraform or Pulumi, is to split "check for drift" from "apply the fix" – `plan()` then `apply()`, the way `terraform plan` and `terraform apply` are two separate commands. `gitops_reconciler` considered this and rejected it, for a reason that's almost tautological once it's stated plainly: all three cloud backends are idempotent by construction, which means a full diff against reality is *inherent* to what `apply()` already has to do before it decides whether to change anything. A no-op tick costs exactly the same diff work whether the wrapper calls `plan()` and skips `apply()`, or just calls `apply()` and lets it discover there's nothing to do. Splitting one method into two doesn't save any work. It adds a second call site and makes the wrapper responsible for a decision the backend was always going to make correctly on its own.

The same reasoning decides where "refresh" logic lives. Terraform and Pulumi both maintain external state that can go stale relative to reality – that's specifically what `--refresh` corrects. CloudFormation has no such artifact; AWS itself is the live state, queried fresh on every call, so "refresh" has nothing to reconcile against for that backend. Putting a `refresh()` method on the shared interface would be meaningful for two backends and a permanent no-op for the third – a leaky abstraction, forcing every implementer to answer a question one of them structurally can't. Keeping refresh-or-not entirely inside each backend's own `apply()` means the interface never asks a question it already knows some answer will be "not applicable."

Why there's no built-in approval gate – and where it actually goes

A fully autonomous reconciler, no human in the loop by default, is a deliberate choice. Staging is where a backend's behavior gets vetted before it's trusted to run unattended against anything real; logs after the fact are enough for post-hoc review once that trust is earned. If a specific backend genuinely needs a review gate, the answer isn't a new method on `BackEnd` – it's a constructor argument on that one backend: `TerraformBackend(dry_run=True)`. Most backends and most ticks don't need a gate at all, and an interface shouldn't carry a capability that only one implementer ever uses.

That's worth stating plainly because it resolves something that could otherwise look like the project quietly reversing itself: the README lists "dry-run mode" under recommended future enhancements, right next to replacing the Pulumi backend's subprocess calls with the real Automation API. Read alongside the constructor-flag reasoning above, that's not a contradiction – it's the same position, followed through. A per-backend `dry_run` flag was always where a review gate belonged. Adding one later is building the thing the design already pointed at, not walking it back.

Why pull-vs-push and convergence stay on separate sides

Two more boundaries, and both come from the same rule: each decision belongs to exactly one layer. *When* to run, *how often*, and *what* triggers a run – a cron tick, a webhook, a person invoking the script by hand – are questions a backend has no business answering; it doesn't know and shouldn't need to. *How* to detect and achieve convergence is the opposite – a question the wrapper has no business answering, because that's what `apply()` exists for. Git sync lives in the wrapper for the same reason: pulling the repo, checking out the right commit, deciding whether anything changed since last sync, is identical work regardless of which backend gets called next, so it happens once, centrally, instead of once per backend implementation.

Locking follows the same discipline at a smaller scale. A single lock around the whole wrapper process would mean one slow Terraform apply blocks an unrelated Pi tick that shares no state with it at all. Scoping the lock to `(backend_name, target)` instead means independent targets tick independently, and `fcntl.flock` self-releases if the process dies, so there's no stale-lock cleanup logic anywhere to get wrong.

What actually ran, and what's sketched

Worth being as plain about this as Chapter 6 was about a bare StatefulSet's guarantees stopping at identity and storage: not every backend in this reconciler has been run against something real in this book. `ComposeBackend` is the one Chapter 19 actually exercised, live – real containers, a real port change, a real revert. `PiBackend` is genuinely implemented, not a stub, but nothing in this book ran it against an actual Raspberry Pi; the same is true of `TerraformBackend` and `PulumiBackend`, both real subprocess-driven implementations that shell out to real CLIs, untested here for lack of cloud infrastructure to point them at. `CloudFormationBackend` is a different case entirely – an honest, admitted stub:

```
def apply(self) -> Status:
    return Status(result=ApplyResult.NO_CHANGE, message="stub: implement via boto3")
```

Its own test doesn't pretend otherwise – it asserts the word "stub" shows up in the message. Eighty-two tests pass across this codebase, 85% statement coverage, and that's a real, healthy number – but it's a number about whether the code behaves the way its own tests say it should, not a claim that every backend has been proven against a real Terraform state file or a real CloudFormation stack. The three methods on `BackEnd` are the same shape whether the implementation behind them is battle-tested or sketched. Knowing which is which, for any given target, is part of trusting the system – not a footnote to leave out because it's less flattering than pretending everything's equally proven.

Chapter 21

Credentials and Blast Radius

The reconciler's environment is the actual security boundary

`TerraformBackend.apply()` shells out to `terraform apply -auto-approve`. `PulumiBackend.apply()` shells out to `pulumi up --refresh -y`. Neither one does anything with credentials – no explicit AWS keys, no assumed role, nothing passed in and nothing read out. That's not an oversight to fix later. `subprocess.run`, called with no environment override, hands the child process the parent's entire environment, and that's the whole credential story for every cloud backend this reconciler has: whatever AWS access is ambient in the shell the wrapper process runs in is exactly the access `terraform` and `pulumi` get when the wrapper calls them. The reconciler doesn't scope, narrow, or manage that access at all. It inherits it, in full, every tick.

That makes the question "where does this process run" the actual security boundary, not a detail beneath one. If the wrapper runs on the same machine it manages, and that machine gets compromised, the attacker doesn't just have the machine – they have whatever credentials let the wrapper reprovision infrastructure from that machine, because those credentials were sitting in the environment the whole time, available to anything running there. Running the reconciler on a separate control machine turns that into a much narrower problem: compromising the target gets an attacker the target, not a standing set of credentials that can also touch everything else the wrapper manages.

Short-lived over static, for the same reason Chapter 12 gave

The other half of limiting blast radius is bounding how long a leaked credential stays useful. A static, long-lived AWS access key that leaks is valid until someone notices and manually revokes it – hours, days, sometimes longer. Short-lived STS credentials, refreshed automatically from an instance role or a federated identity, expire on their own, typically within an hour. A credential that leaks and expires forty minutes later is a smaller incident than one that stays live until a human catches it. Preferring STS-style credentials over static keys "wherever the provider supports it" isn't a preference for its own sake – it's shrinking the same window `PulumiBackend`'s `--refresh` flag exists to keep honest, applied to the credential layer instead of the state layer: don't let something stay trusted longer than it has to.

The exception that proves the rule

`PiBackend` breaks this pattern entirely, on purpose, and the code shows exactly why the pattern doesn't apply there. Look at how it decides whether to use SSH at all:

```
def _ssh_prefix(self) -> list[str]:
    return [] if self._cfg.host in ("", "local") else ["ssh", self._cfg.host]
```

No cloud API, no assumed role, no STS anywhere in this backend – `apply()` either runs the configured command directly, on the box it's already running on, or reaches one specific Pi over SSH. There's no meaningful privilege boundary between "the wrapper" and "the thing it manages" to protect in either case: a Raspberry Pi on a home LAN managing itself has no blast radius bigger than the Pi. Running the reconciler loop on the Pi it manages – something that would be a real mistake for a Terraform-backed target holding AWS credentials – is simply fine here, because there's nothing broader for a compromised Pi to reach that the reconciler's presence made newly reachable. The rule about separate control machines exists specifically to protect credentials broader than the target itself. Where there's no such credential, there's nothing the rule is protecting.

The same split as Chapter 12, one layer up

This chapter's actual question – who can make the reconciler act against a given target – is the same shape Chapter 12 drew between application auth and cluster RBAC, just moved one level up the stack. There, the split was "who can call `toy-api`'s endpoints" versus "who can run `kubectl apply` against the cluster itself," two unrelated questions enforced by different code at different layers. Here it's "who can get code executed as the reconciler process" – and therefore inherit whatever credentials sit in that process's environment – versus "who can write to the git repo the reconciler watches," which is a completely separate access-control question, answered by git hosting permissions, not by anything in `gitops_reconciler` itself. A person with commit access to the watched repo can get arbitrary Terraform or Compose config applied on the next tick, without ever touching the machine the wrapper runs on. A person with shell access to the wrapper's control machine has the ambient cloud credentials directly, without needing to touch git at all. Neither access implies the other, and mixing them up here would be the same mistake Chapter 12 spent a whole chapter clearing up – just with "git write access" standing in for "application auth," and "shell access to the control machine" standing in for "cluster RBAC."

Chapter 22

Progressive Delivery Without New Abstractions

Two targets, one repo, no new machinery

Staging tracks `:latest`. Production pins to a specific, already-validated tag. Both watch the same git repo – `example-app/` – and apply different compose files inside it:

```
# docker-compose.staging.yml
services:
  demo-app:
    image: gitops-demo-app:latest
```

```
# docker-compose.prod.yml
services:
  demo-app:
    image: gitops-demo-app:v1.0.0
```

That's the entire trick, and it's worth stating plainly: nothing about `ManagedTarget` or `tick()` changed to support this. Both targets are ordinary `ComposeBackend` instances, each with its own lock file and state file, each reconciling on its own schedule against its own `docker-compose.*.yml`. Progressive delivery here isn't a feature the reconciler had to grow. It's two instances of a pattern that already existed, pointed at two files that happen to differ in one line.

Watching a real promotion happen

Start staging, applying `latest` for real:

```
python -m gitops_reconciler.example --target demo-app-staging
```

```
demo-app-staging: changed
```

Check what got recorded:

```
cat /tmp/gitops-agent/state/demo-app-staging.json
```

```
{"sha": "908c552f5be93bf2cb459c5a86b8766c48dfb280", "result": "changed", "message": ""}
```

That's `record_last_sha()` from Chapter 20's wrapper – the same provenance mechanism that exists purely for audit-trail reasons in the base design – turning out to be the load-bearing piece of a completely different, more sophisticated pattern without anyone having to add anything for it. `promote.py`, copied from `promote.example.py`, reads exactly that file:

```
./promote.py --dry-run
```

```
Staging last applied: 908c552
Production current pin: v1.0.0
[DRY RUN] Would update .../docker-compose.prod.yml:
    image: gitops-demo-app:908c552
```

Run it for real, commit the result, and prod's next tick picks it up the same way every other change in this book has propagated – through git, not through the promotion script touching prod directly:

```
./promote.py
git add example-app/docker-compose.prod.yml
git commit -m "Promote demo-app to staging SHA 908c552"
git push
```

```
python -m gitops_reconciler.example --target demo-app-prod
```

```
demo-app-prod: changed
```

```
docker compose -f example-app/docker-compose.prod.yml -p gitops-demo-prod ps
```

```
NAME                           IMAGE                       STATUS
gitops-demo-prod-demo-app-1    gitops-demo-app:908c552     Up
```

Prod is now running the exact SHA staging validated. `promote.py` never touched prod, and prod's reconciler never talked to staging's – the promotion script read one state file and wrote one compose file, and the rest was the ordinary reconciliation loop this whole book has been watching, doing what it always does.

Two parallels, not one

This is Chapter 14's drift detection with a human-gated promotion step inserted where `selfHeal` would otherwise fire automatically – staging auto-applies on every change the way a `selfHeal: true` Application does, and prod stays pinned until a human, not a controller, decides it's time to move the pin forward. But it's also Chapter 15's problem, solved with the same tool aimed differently. Chapter 15 closed by naming the failure mode of hand-copied per-environment YAML plainly: nothing ties `staging/deployment.yaml` and `prod/deployment.yaml` together once they're copied, so they drift apart from *each other* over time with nothing watching for it. `record_last_sha` and `last_recorded_sha` are that missing tie – one shared provenance field connecting two independently-reconciled targets, so the pin can't silently drift out of sync with what staging actually validated. Chapter 15 solved its version with one generator producing many environments from one template. This solves it with one recorded fact two environments both read from – different shape, same underlying complaint: nothing should be allowed to drift apart from what it's supposed to track without someone noticing.

The honest gap, demonstrated

`last_recorded_sha` answers "what SHA did staging last successfully apply." It does not answer "is that still what's actually running on staging right now," and the difference is easy to miss until it's demonstrated directly. Stop staging's container by hand, without touching git and without running another tick:

```
docker compose -f example-app/docker-compose.staging.yml \
  -p gitops-demo-staging stop
```

The state file doesn't change – nothing ran to change it:

```
cat /tmp/gitops-agent/state/demo-app-staging.json
```

```
{"sha": "908c552f5be93bf2cb459c5a86b8766c48dfb280", "result": "changed", "message": ""}
```

Ask `promote.py` what it thinks, with staging's container actually stopped:

```
./promote.py --dry-run
```

```
Staging last applied: 908c552
Production current pin: 908c552
Production is already at staging's SHA, no promotion needed
```

Perfectly reasonable, given what the script can see – and quietly wrong. Staging isn't running at all right now, and `promote.py` has no way to know that, because it never checks staging's live state, only the state file recorded the last time a tick actually succeeded there. ArgoCD's `selfHeal` closed a version of this exact gap in Chapter 14 – continuously re-checking live cluster state against git, not trusting a stale record of what *used* to be true. Nothing here plays that role. If staging drifted, crashed, or got hand-patched sometime after its last successful tick, promotion would happily ship whatever SHA that old tick recorded, on the reasonable-sounding but false assumption that a past success is still a present one. The fix, if it mattered enough to build, would be exactly what Chapter 14 already demonstrated: re-tick staging immediately before trusting its recorded SHA, the same continuous re-checking that

makes `selfHeal` trustworthy instead of just fast. Nothing in this reconciler does that automatically, and that's worth knowing rather than assuming away.

Chapter 23

One Pattern, Three Substrates

The same three verbs, every time

Twenty-two chapters, three genuinely different pieces of software – Docker Compose, Kubernetes plus ArgoCD, and a hand-rolled Python reconciler – and every single one of them turned out to be the same loop, wearing different clothes:

```
loop forever:
    desired = read the declared state
    actual  = observe the real world
    if desired != actual:
        act to close the gap
```

Compose's version of this loop only runs once, on demand – `docker compose up` reads the file, checks the running containers, and converges, then stops. Kubernetes' version runs continuously and lives inside the cluster, watching etcd instead of a file on disk, with ArgoCD adding one more layer of the identical loop on top, watching git instead of a person's `kubectl` history. The reconciler from Part V builds that same loop from nothing, for targets that never had a Kubernetes-style control plane sitting underneath them to begin with. Declare, observe, reconcile. The verbs never changed. What changed, chapter to chapter, was only ever where the loop lived, how often it ran, and what "observe" and "act" actually meant for the thing being managed.

What each substrate actually buys, side by side

	Docker Compose	Kubernetes + ArgoCD	`gitops_reconciler`
Where the loop runs	On demand, when you run it	Continuously, inside the cluster	Continuously, wherever the wrapper process runs
What it observes	The current host's containers	Cluster state via the API server	Whatever `get_outputs()`/a hash comparison can see
Multi-host scheduling	No – Chapter 4's real ceiling	Yes – the scheduler's whole job	No – one target, one apply mechanism
Self-healing	No, not automatically	Yes, standing order (Ch. 7)	Only if the backend's own tool provides it
Drift detection	No	Yes, continuous and visible (Ch. 14)	Only as good as the last recorded tick (Ch. 22)
Setup cost	Minutes	Hours, plus real ongoing cost (Ch. 9, 17)	An afternoon, per new backend
What "idempotent" costs	Nothing native – Compose just reapplies	Native, built into every controller	Depends on backend: free for cloud tools, faked with a hash for Compose/Pi (Ch. 20)
Where it fits	One host, one team	Multiple hosts, fleet-scale operations	Anything with no built-in control plane at all

None of these rows is a verdict. Compose scoring "no" on multi-host scheduling isn't a defect – Chapter 4 already made the case that this is exactly the right tradeoff for a workload that only ever needed one host. The table is a map of what each substrate is actually for, read the way Chapter 9 and Chapter 17 both insisted it be read: against a specific workload's specific needs, not as a ranking with Kubernetes at the top.

A checklist for a new project

Pulled together from Chapter 9's signals and Chapter 17's framework, because they were always asking the same question at two different layers – deployment substrate and autoscaling substrate – with the same answer both times: match the tool to the need that's actually present, not the one that might show up eventually.

- **Does this run on one host, for one team, right now?** If yes, and none of the signals below are true yet, Compose (or the equivalent single-box pattern from Chapter 17) is the right answer, not a placeholder for something more serious.
- **Has a specific, concrete trigger actually happened** – one host genuinely out of headroom, a deploy that has to happen without an outage because people are actively using the thing, a second team that needs to ship independently, a hardware failure that took something down for real? Chapter 9's argument was that these are the actual signals, not a vague sense that the project has gotten "serious." Wait for one of them, not for a feeling.
- **Is the fixed cost smaller than what it buys?** Kubernetes and a real EKS cluster both have costs that don't scale down with idle time – $73/month for a control plane whether or not anything's running in it. That's fine once the workload justifies it, and a bad deal for everything below that line.
- **Does the target already have a control plane, or does one need to be built?** Kubernetes supplies etcd and an API server for free. A Terraform-managed cloud stack, a Compose host, a Raspberry Pi – none of them do, which is the entire reason Part V's reconciler exists. Reaching for ArgoCD against a target with no Kubernetes underneath it is reaching for the wrong half of the pattern; reaching for a hand-rolled reconciler against something Kubernetes already manages is rebuilding what's already there for free.
- **What's the actual failure mode being protected against?** Chapter 22 named the sharpest version of this: `selfHeal` and `last_recorded_sha` look like they're solving the same problem, but only one of them continuously re-checks live reality instead of trusting a past success. Know which guarantee a given substrate actually gives before assuming it gives the stronger one.

Answer these honestly for a real project, and the substrate mostly picks itself – the same way it did, chapter by chapter, across every one of the twenty-two before this one.

Chapter 24

What Still Isn't Covered

The honest list

Chapter 2 set a principle early and asked the rest of this book to carry it: if it's not in git, it shouldn't be running. Twenty-three chapters later, that principle has been kept – but it was kept for a deliberately narrow slice of what a real production system actually needs. Naming what's outside that slice, plainly, is more useful than letting the book's silence on a topic pass as "solved" or "unimportant."

Secrets management at scale. `postgres-secret.yaml` and `api-key-secret.yaml` in this book both said the same thing in their own comments: fine for a learning exercise, base64 is not encryption, real deployments need Sealed Secrets or External Secrets Operator pulling from an actual vault. Chapter 8 and Chapter 12 both demonstrated the *shape* of a Secret – masked in `kubectl describe`, separate from ConfigMaps – and neither one built the hardened version. SOPS, encrypting secrets in git directly rather than keeping git free of them entirely, is the other common answer and wasn't touched at all.

A full observability stack. Chapter 11 built real structured logging and proved, live, that `kubectl logs` loses a pod's history the moment the pod is gone – and then stopped exactly at the point where Loki or the EFK stack would actually solve that. `app.py` exposes a `/metrics` endpoint via `prometheus_fastapi_instrumentator`, and Chapter 10's Pulumi program deploys an actual Prometheus and Grafana pair for a different app entirely – but nothing in this book wired the toy API's own metrics into a dashboard, set up an alert, or traced a request across more than one service. The gap Chapter 11 demonstrated is real and still open.

RBAC and network policy, past the basics. Chapter 12 covered the distinction between application auth and cluster RBAC, and Chapter 21 carried the same split one layer up into the reconciler – but neither one built a real least-privilege Role for a production workload, and NetworkPolicy objects, restricting which pods can even talk to which other pods over the network, never came up at all. A cluster with no NetworkPolicies is one where every pod can reach every other pod by default, regardless of how carefully RBAC is scoped.

Image scanning and supply-chain attestation. Chapter 2 argued that bad actors have industrialized – automated scanning, supply-chain attacks – as part of the case for version-controlled infrastructure in the first place. This book never closed the loop on the container image side of that same argument: nothing here scans `k8s-toy-api:local` for known vulnerabilities before it runs, and nothing verifies the image running in the cluster is actually the image that was built from reviewed source, the way SBOM generation and image signing (cosign, Sigstore) are meant to guarantee.

Where to actually go next

None of these are exotic. Each one is a natural continuation of a chapter that's already been read, not a new subject dropped in cold – External Secrets Operator extends Chapter 8's ConfigMap/Secret split; Loki extends Chapter 11's structured logging; a real least-privilege Role extends Chapter 12's ServiceAccount work; image scanning extends Chapter 2's own argument about industrialized threats back to where it started. Chapter 2's principle still holds as the filter for evaluating any of them: if a secret, a metric, a permission, or an image's provenance isn't recorded somewhere auditable, it doesn't really exist as far as the system's security posture is concerned, no matter how carefully everything upstream of it was built. This book got the pattern right – declare, observe, reconcile – across three substrates. Getting the pattern right and closing every gap in what it's applied to were always two different jobs, and only the first one was this book's.

Appendix A

Command Reference

Every command below was actually run somewhere in this book. The chapter number is where to find the fuller transcript and explanation.

Docker / Docker Compose

Command	What it does	Chapter
`docker build -t <tag> .`	Build an image from a Dockerfile	3, 6
`docker history <image>`	Show an image's layer stack and sizes	3
`docker compose up -d --build`	Build and start all services in the background	4, 19
`docker compose ps -a`	List all containers for a project, including stopped	4, 19, 22
`docker compose -f <file> -p <project> ps`	List containers for a specific compose file/project pair	19, 22
`docker compose up -d --scale <svc>=N`	Try to run N replicas of one service	4
`docker compose down`	Stop and remove a project's containers and network	4, 19

kubectl: pods, deployments, and workloads

Command	What it does	Chapter
`kubectl get pods -l app=<label>`	List pods matching a label selector	6, 7, 12, 13, 16
`kubectl get pods -w`	Watch pod status changes live	7
`kubectl delete pod <name>`	Delete a pod; its controller replaces it	6, 7, 8, 13, 14, 22
`kubectl scale deployment/<name> --replicas=N`	Change a Deployment's desired replica count	7, 13, 14
`kubectl scale statefulset/<name> --replicas=N`	Same, for a StatefulSet	7
`kubectl exec <pod> -- <cmd>`	Run a command inside a running container	7, 12, 16, 22
`kubectl wait --for=condition=ready pod -l <label>`	Block until matching pods report ready	6, 7, 8

Command	What it does	Chapter
`kubectl rollout restart deployment/<name>`	Trigger a rolling restart	8
`kubectl rollout status deployment/<name>`	Watch a rollout until it completes or fails	8
`kubectl logs <pod>` / `kubectl logs -l <label>`	Fetch a container's stdout/stderr	8, 11
`kubectl describe pod <name>`	Full pod detail, including env var provenance	8, 12

kubectl: config, secrets, RBAC

Command	What it does	Chapter
`kubectl get configmap <name> -o yaml`	Show a ConfigMap's current contents	8
`kubectl patch configmap <name> --type merge -p '<json>'`	Edit a ConfigMap in place	8
`kubectl get secret <name> -o yaml`	Show a Secret (base64-encoded, not decrypted)	8
`kubectl auth can-i <verb> <resource> [--as=<identity>]`	Check whether an identity can perform an action	12
`kubectl auth can-i --list`	List everything the current identity can do	12

kubectl: ArgoCD, KEDA, and cluster inspection

Command	What it does	Chapter
`kubectl get application <name> -n argocd`	Check an ArgoCD Application's sync/health status	13, 14
`kubectl get applications -n argocd`	List all ArgoCD Applications	13, 15
`kubectl patch application <name> -n argocd --type merge -p '<json>'`	Force a refresh, change syncPolicy, or trigger a sync	13, 14
`kubectl get hpa`	List HorizontalPodAutoscalers, including KEDA-managed ones	16
`kubectl get deployment <name>`	Check a Deployment's ready/available replica counts	16
`kubectl get pods -n kube-system`	See the control plane's own pods	5

Git

Command	What it does	Chapter
`git commit -am "<message>"`	Commit all tracked changes with a message	13, 22
`git push <remote> <branch>`	Push commits to a remote	13, 22

Pulumi

Command	What it does	Chapter
`pulumi preview`	Show what `pulumi up` would change, without applying	10
`pulumi up`	Apply the program's desired state	10
`pulumi destroy`	Tear down everything the stack manages	10
`pulumi stack output <name>`	Print one exported output value	10

gitops_reconciler

Command	What it does	Chapter
`uv run python reconcile_example.py`	Run one reconciliation tick against the demo Compose stack	19
`./watch_and_reconcile.sh [interval]`	Run ticks on a loop, polling git for changes	19
`python -m gitops_reconciler.example --target <name>`	Run one real tick (with git sync) against a named target	22
`./promote.py [--dry-run]`	Promote staging's last-applied SHA to production's pin	22

Appendix B

Glossary

ApplicationSet – an ArgoCD object that generates multiple `Application` objects from one template and a generator (e.g., a directory listing). Chapter 15.

BackEnd (ABC) – the abstract base class every `gitops_reconciler` backend implements: `apply()`, `destroy()`, `get_outputs()`. Chosen over a `Protocol` because Pydantic gives it a real `isinstance()` check. Chapters 19-20.

Control loop – the pattern underlying every reconciliation system in this book: read desired state, observe actual state, act to close the gap, repeat forever. Chapter 5 and onward.

ConfigMap – a Kubernetes object holding non-secret configuration, injected into pods as environment variables or mounted files. Chapter 8.

Desired state – what a system is declared to look like, as opposed to what it currently looks like (actual state). The gap between the two is what every controller in this book exists to close.

Drift – when actual state no longer matches desired state, usually because something changed it directly rather than through the declared source of truth. Chapters 1, 14, 22.

GitOps – managing infrastructure by treating a git repository as the source of truth and running a controller that continuously reconciles real state to match it. Chapters 13-22.

HPA (HorizontalPodAutoscaler) – the Kubernetes object that scales a Deployment's replica count based on a metric. KEDA creates and drives a real HPA rather than replacing it. Chapter 16.

Idempotent – an operation that produces the same result whether run once or many times. `apply()` is supposed to be idempotent for every backend in `gitops_reconciler`; Compose and Pi fake it with a hash comparison since they have no native diff. Chapter 20.

IRSA (IAM Roles for Service Accounts) – the AWS mechanism binding a Kubernetes ServiceAccount to an IAM role via OIDC federation, so pods get scoped AWS credentials instead of inheriting the node's. Chapter 12.

KEDA (Kubernetes Event-Driven Autoscaling) – scales workloads based on external metrics (queue depth, etc.) rather than CPU/memory alone, by feeding a custom metric to a standard HPA. Chapter 16.

Operator – a controller, following the same watch-diff-act pattern as everything built into Kubernetes, that encodes domain-specific operational knowledge (e.g., how to run Postgres) on top of primitives like StatefulSet. Chapters 6, 9, 12.

Provenance (in this book's reconciler) – the git SHA recorded after a successful `apply()` call. The mechanism that makes Chapter 22's promotion pattern possible without any new abstractions.

Reconciliation – the act of comparing desired and actual state and acting to close any gap. The verb behind every noun in this glossary.

RBAC (Role-Based Access Control) – Kubernetes' system for controlling which identities can perform which actions against the API server, via Roles and RoleBindings (or ClusterRole/ClusterRoleBinding). Distinct from application-level auth. Chapter 12.

Secret – a Kubernetes object like ConfigMap, but for sensitive values – base64-encoded (not encrypted) by default, masked in `kubectl describe` output. Chapter 8.

selfHeal – an ArgoCD Application syncPolicy setting that automatically reverts manual changes to match git, rather than just flagging them as `OutOfSync`. Chapter 14.

ServiceAccount – the identity a pod runs as when it talks to the Kubernetes API. Defaults to `default` with no permissions unless a Role is explicitly bound to it. Chapters 12, 21.

StatefulSet – a Kubernetes controller for workloads needing stable identity and stable storage across rescheduling. Guarantees stop there – no built-in replication, failover, or backups. Chapter 6.

Appendix C

Repository Map

This book draws on three repositories, each anchoring a different part.

Repo	Anchors	What it actually is
`k8s-hack`	Parts I-IV (Chapters 1-18)	This book's own source, plus the toy API and Kubernetes manifests every hands-on chapter through Chapter 18 walks through directly
`gitops-lab`	Part IV (Chapters 13-18)	A working kind + ArgoCD + Gitea setup: the `k8s-hack` Application, the `gitops-lab-envs` ApplicationSet, and the KEDA/RabbitMQ demo, all live and referenced with real command output
`gitops_reconciler`	Part V (Chapters 19-22)	The backend-agnostic reconciler itself – `BackEnd`, `ManagedTarget`, `tick()` – plus the Compose demo and the staging/prod promotion example

Suggested reading order, if not reading start to finish: Parts I-III (Chapters 1-12) stand alone as a Kubernetes fundamentals course and don't require either of the other two repos. Part IV (13-18) needs `gitops-lab` running to reproduce the transcripts, but its concepts build directly on Part III and shouldn't be read out of order relative to it. Part V (19-22) needs `gitops_reconciler` and stands mostly independent of Parts III-IV conceptually – someone who only cares about GitOps outside Kubernetes could reasonably start at Chapter 19 after reading Chapters 1-2 and 5 for the control-loop framing, though the cross-references back to Chapters 12 and 14 will land better having read those first. Part VI (23-24) and the appendices assume everything before them.